TỔNG LIÊN ĐOÀN LAO ĐỘNG VIỆT NAM
TRƯỜNG ĐẠI HỌC TÔN ĐỨC THẮNG
KHOA CÔNG NGHỆ THÔNG TIN



VÕ ANH KIỆT - 523H0048

ĐỒ ÁN TỐT NGHIỆP

PHÁT TRIỂN NỀN TẢNG KHÔNG GIAN LÀM VIỆC CỘNG TÁC THỜI GIAN THỰC TÍCH HỢP HỘI NGHỊ TRUYỀN HÌNH ĐA ĐIỂM

THÀNH PHỐ HỒ CHÍ MINH, NĂM 2026

TỔNG LIÊN ĐOÀN LAO ĐỘNG VIỆT NAM
TRƯỜNG ĐẠI HỌC TÔN ĐỨC THẮNG
KHOA CÔNG NGHỆ THÔNG TIN



VÕ ANH KIỆT - 523H0048

ĐỒ ÁN TỐT NGHIỆP

PHÁT TRIỂN NỀN TẢNG KHÔNG GIAN LÀM VIỆC CỘNG TÁC THỜI GIAN THỰC TÍCH HỢP HỘI NGHỊ TRUYỀN HÌNH ĐA ĐIỂM

Người hướng dẫn
Th.S Phù Trần Tín

THÀNH PHỐ HỒ CHÍ MINH, NĂM 2026

LỜI CẢM ƠN

Em xin được gửi lời cảm ơn chân thành đến thầy Phù Trần Tín. Thầy là người đã hướng dẫn chính và hỗ trợ em trong bài tiểu luận này, em đã nhận được rất nhiều sự quan tâm, giúp đỡ đến từ thầy. Nhờ vào đó, em đã vận dụng những kiến thức đã được học và tìm tòi thêm những kiến thức mới để có thể hoàn thành bài tiểu luận của mình. Tuy nhiên, do hạn chế về kinh nghiệm và kiến thức còn hạn hẹp nên không thể tránh khỏi những thiếu sót trong bài làm. Em rất mong rằng bài tiểu luận này có thể thuyết phục được thầy/cô là giảng viên chấm bài. Em xin chân thành cảm ơn ạ!

TP. Hồ Chí Minh, tháng 10 năm 2026

Tác giả

(Ký và ghi rõ họ tên)

Võ Anh Kiệt

LỜI CAM ĐOAN

Em xin cam đoan đây là công trình nghiên cứu của riêng em và được sự hướng dẫn khoa học của Th.S. Phù Trần Tín. Các nội dung nghiên cứu, kết quả trong đề tài này là trung thực và chưa công bố dưới bất kỳ hình thức nào trước đây. Những số liệu trong các bảng biểu phục vụ cho việc phân tích, nhận xét, đánh giá được chính tác giả thu thập từ các nguồn khác nhau có ghi rõ trong phần tài liệu tham khảo.

Ngoài ra, trong Dự án còn sử dụng một số nhận xét, đánh giá cũng như số liệu của các tác giả khác, cơ quan tổ chức khác đều có trích dẫn và chú thích nguồn gốc.

Nếu phát hiện có bất kỳ sự gian lận nào em xin hoàn toàn chịu trách nhiệm về nội dung Dự án của mình. Trường Đại học Tôn Đức Thắng không liên quan đến những vi phạm tác quyền, bản quyền do tôi gây ra trong quá trình thực hiện (nếu có).

TP. Hồ Chí Minh, tháng 10 năm 2026

Tác giả

(Ký và ghi rõ họ tên)

Võ Anh Kiệt

TÓM TẮT

Nhóm làm việc từ xa, lớp học trực tuyến và các cộng đồng trên mạng thường phải dùng song song nhiều công cụ: một ứng dụng nhắn tin cho trao đổi hằng ngày, một dịch vụ khác cho họp video và một công cụ nữa để giao việc. Mỗi lần chuyển công cụ là một lần đứt mạch ngữ cảnh. Đồ án này thiết kế, hiện thực và đánh giá **nexo**, một không gian cộng tác chạy trên trình duyệt, kết hợp nhắn tin thời gian thực theo kênh, trạng thái hiện diện, hội nghị truyền hình đa điểm và quản lý công việc trong cùng một ứng dụng, tổ chức theo mô hình *Workspace → Channel*.

Hệ thống gồm một client đơn trang viết bằng React/TypeScript, một máy chủ ứng dụng Node.js đảm nhận REST, nhắn tin qua Socket.IO và signaling WebRTC, một máy chủ định tuyến chọn lọc (SFU) mediasoup với nhóm worker C++, và cơ sở dữ liệu PostgreSQL vốn còn được dùng để đồng bộ sự kiện và trạng thái hiện diện giữa các tiến trình ứng dụng. Tin nhắn được gửi theo một giao thức lũy đẳng (mã tin do client sinh, outbox bền vững ở client và ràng buộc duy nhất trong cơ sở dữ liệu), nhờ đó việc mất xác nhận hay kết nối lại không làm mất hoặc nhân đôi tin. Client hội nghị dùng simulcast VP8 ba lớp và một lưới video động chỉ yêu cầu những lớp mà nó thực sự hiển thị.

Hệ thống được đánh giá bằng kiểm thử tự động, một bài kiểm tra tải WebSocket và một phòng thí nghiệm media, trong đó mỗi người tham gia là một tiến trình Chromium headless riêng. Với 200 socket đồng thời, độ trễ nhận tin ở phân vị 95 luôn dưới 50 ms, không có tin bị mất hay bị trùng. So với một hiện thực Mesh ngang hàng toàn phần được dùng làm hệ thống chuẩn trên cùng phần cứng, SFU giữ uplink của mỗi client gần như không đổi khi phòng đông lên, trong khi uplink và CPU của client Mesh tăng tuyến tính và phiên Mesh sập ở sáu người. Mọi thực nghiệm đều chạy trên một máy qua mạng nội bộ; việc đo trên nhiều mạng, qua TURN và ở quy mô triển khai thật là hướng phát triển tiếp theo.

Từ khóa: cộng tác thời gian thực, WebSocket, Socket.IO, WebRTC, SFU, mediasoup, simulcast, QoS.

MỤC LỤC

CHƯƠNG 1. TỔNG QUAN ĐỀ TÀI 1

1.1 Lý do chọn đề tài 1

1.2 Vấn đề cần giải quyết 1

1.3 Mục tiêu 2

1.4 Phạm vi 2

1.4.1 Đối tượng sử dụng 2

1.4.2 Trong phạm vi 2

1.4.3 Ngoài phạm vi 3

1.5 Phương pháp thực hiện 3

1.6 Công nghệ sử dụng 4

1.7 Bố cục báo cáo 4

CHƯƠNG 2. CƠ SỞ LÝ THUYẾT 6

2.1 Giao tiếp thời gian thực trên web 6

2.1.1 Từ mô hình yêu cầu – phản hồi đến WebSocket 6

2.1.2 Socket.IO 7

2.1.3 Đồng bộ trạng thái hiện diện 7

2.2 Công nghệ WebRTC 8

2.2.1 Thương lượng phiên bằng SDP 8

2.2.2 Vượt NAT: ICE, STUN và TURN 8

2.2.3 Bảo mật: DTLS-SRTP 9

2.2.4 RTP, RTCP và các đại lượng QoS 10

2.2.5 Điều khiển tắc nghẽn, simulcast và codec 10

2.3 Các mô hình hội nghị đa điểm 11
2.3.1 Mesh *11*
2.3.2 MCU *11*
2.3.3 SFU *11*
2.3.4 So sánh định lượng *12*
2.4 Thư viện SFU mediasoup 13
2.5 Kiểm soát truy cập theo vai trò 13
2.6 Tóm tắt chương 14
CHƯƠNG 3. SƠ ĐỒ KHỐI VÀ THIẾT KẾ HỆ THỐNG 15
3.1 Phân tích yêu cầu 15
3.1.1 Tác nhân *15*
3.1.2 Yêu cầu chức năng *15*
3.1.3 Yêu cầu phi chức năng *16*
3.1.4 Ca sử dụng chính *17*
3.2 Sơ đồ khối hệ thống 17
3.2.1 Tổng quan *17*
3.2.2 Tách luồng điều khiển và luồng dữ liệu *19*
3.3 Phân tích từng khối 19
3.3.1 Khối client *19*
3.3.2 Khối máy chủ ứng dụng và signaling *20*
3.3.3 Khối media SFU *20*
3.3.4 Khối dữ liệu *21*
3.3.5 Khối vượt NAT *21*

3.4 Thiết kế cơ sở dữ liệu 21
3.5 Thiết kế phân quyền 23
3.6 Thiết kế giao thức thời gian thực 24
3.6.1 Danh mục sự kiện 24
3.6.2 Luồng gửi tin nhắn 24
3.6.3 Tham gia kênh video 25
3.7 Thiết kế mở rộng ngang 26
3.8 Thiết kế giao diện 27
3.9 Tóm tắt chương 27
CHƯƠNG 4. TRIỂN KHAI HỆ THỐNG 28
4.1 Tổ chức mã nguồn 28
4.2 Lõi thời gian thực 29
4.2.1 Xác thực và kiểm quyền từng sự kiện 29
4.2.2 Lưu tin nhắn đúng một lần 30
4.2.3 Trạng thái hiện diện và đang gõ 32
4.2.4 Phân phối sự kiện giữa nhiều tiến trình 34
4.3 Lõi hội nghị 34
4.3.1 Nhóm worker và phân bổ phòng 34
4.3.2 Vòng đời producer và consumer 35
4.3.3 Phiên họp và điều hành 35
4.3.4 Vượt NAT và phục hồi ICE 35
4.4 Client hội nghị và lưới video động 36
4.4.1 Thu và phát media 36

4.4.2 Thu thập số liệu QoS *37*

4.4.3 Lưới video động và chọn lớp theo nhu cầu *37*

4.5 Giao diện người dùng 39

4.6 Triển khai 43

4.7 Tóm tắt chương 44

CHƯƠNG 5. THỰC NGHIỆM VÀ ĐÁNH GIÁ 45

5.1 Thiết lập thực nghiệm 45

5.1.1 Môi trường và quy mô thử nghiệm *45*

5.1.2 Các đại lượng đo *46*

5.1.3 Hệ thống chuẩn để so sánh *47*

5.2 Kiểm thử chức năng và độ tin cậy 47

5.3 Độ trễ của lõi nhắn tin khi chịu tải 49

5.3.1 Kịch bản *49*

5.3.2 Kết quả *49*

5.4 So sánh Mesh và SFU 51

5.4.1 Thiết kế thực nghiệm *51*

5.4.2 Các sai lệch đo lường *52*

5.4.3 Kết quả: băng thông *53*

5.4.4 Kết quả: CPU *56*

5.4.5 Kết quả: độ trễ và chất lượng *57*

5.4.6 Tác dụng của simulcast và chọn lớp *59*

5.4.7 Thảo luận *60*

5.5 So sánh với các hệ thống hiện có 61

5.6 Đối chiếu với yêu cầu.....62

5.7 Giới hạn của phép đo 63

5.8 Tóm tắt chương 64

CHƯƠNG 6. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN.....65

6.1 Kết quả đạt được so với mục tiêu.....65

6.2 Đóng góp của đồ án.....66

6.3 Hạn chế.....67

6.4 Hướng phát triển 67

TÀI LIỆU THAM KHẢO 69

DANH MỤC CÁC CHỮ VIẾT TẮT

Chữ viết tắt	Ý nghĩa
ACK	Acknowledgement – xác nhận
API	Application Programming Interface – giao diện lập trình ứng dụng
CDP	Chrome DevTools Protocol
CPU	Central Processing Unit – bộ xử lý trung tâm
DTLS	Datagram Transport Layer Security
FPS	Frames per second – số khung hình mỗi giây
GCC	Google Congestion Control
ICE	Interactive Connectivity Establishment
JWT	JSON Web Token
MCU	Multipoint Control Unit
NACK	Negative Acknowledgement – yêu cầu gửi lại gói
NAT	Network Address Translation – biên dịch địa chỉ mạng
P2P	Peer-to-peer – ngang hàng
PLI	Picture Loss Indication – yêu cầu khung khóa
QoS	Quality of Service – chất lượng dịch vụ
RBAC	Role-Based Access Control – phân quyền theo vai trò
REST	Representational State Transfer
RTCP	RTP Control Protocol
RTP	Real-time Transport Protocol
RTT	Round-trip time – thời gian khứ hồi
SDP	Session Description Protocol
SFU	Selective Forwarding Unit – bộ định tuyến chọn lọc
SPA	Single-Page Application – ứng dụng đơn trang
SRTP	Secure Real-time Transport Protocol
SSRC	Synchronisation Source – định danh nguồn
STUN	Session Traversal Utilities for NAT
TTL	Time to live – thời gian sống
TURN	Traversal Using Relays around NAT
UDP / TCP	User Datagram Protocol / Transmission Control Protocol
VP8	Chuẩn mã hóa video định nghĩa trong RFC 6386
WebRTC	Web Real-Time Communication

DANH MỤC HÌNH VẼ

Hình 2.1. Các loại candidate ICE và đường kết nối giữa trình duyệt và SFU 9
Hình 2.2. Ba mô hình Mesh, MCU và SFU với một phòng bốn người 11
Hình 3.1. Các ca sử dụng chính của hệ thống 17
Hình 3.2. Sơ đồ khối hệ thống 18
Hình 3.3. Các thực thể cốt lõi của cơ sở dữ liệu 22
Hình 3.4. Sơ đồ tuần tự gửi tin nhắn tin cậy 25
Hình 3.5. Sơ đồ tuần tự tham gia kênh video 26
Hình 3.6. Mở rộng ngang: nhắn tin dùng chung, media theo room affinity 27
Hình 4.1. Chuỗi kiểm quyền từng sự kiện của cổng Socket.IO 29
Hình 4.2. Lưu đồ gửi tin nhắn lũy đẳng ở máy chủ 31
Hình 4.3. Heartbeat trạng thái hiện diện chạy trên từng tiến trình API mỗi 15 giây 33
Hình 4.4. Sơ đồ trạng thái phục hồi ICE ở client 36
Hình 4.5. Lưới video theo nhu cầu: từ bố cục ô tới yêu cầu gửi SFU 38
Hình 4.6. Trang chủ với công việc được giao, lịch họp sắp tới và thông báo 40
Hình 4.7. Workspace với danh sách kênh và một kênh văn bản 40
Hình 4.8. Kênh video với bảng điều hành và nút chuyển bố cục 41
Hình 4.9. Trang quản trị workspace 42
Hình 4.10. Giao diện chat (trái) và cuộc họp (phải) trên màn hình điện thoại rộng 375 điểm ảnh 43
Hình 4.11. Mô hình triển khai bằng Docker Compose 44
Hình 5.1. Môi trường thực nghiệm media 46
Hình 5.2. Các phân vị độ trễ nhận tin ở mức 50, 100 và 200 socket 50
Hình 5.3. Uplink của mỗi client theo số người tham gia 54
Hình 5.4. Downlink của mỗi client theo số người tham gia 55
Hình 5.5. CPU trình duyệt của mỗi client theo số người tham gia 56
Hình 5.6. Thời gian khứ hồi theo số người tham gia (thang logarit) 57
Hình 5.7. Tốc độ khung hình nhận theo số người tham gia 58
Hình 5.8. Downlink của mỗi client với ba cấu hình SFU ở N = 6 và N = 10 (cấu hình một bản mã hóa chỉ đo tới N = 8) 60

DANH MỤC BẢNG BIỂU

Bảng 1.1. Công nghệ sử dụng trong hệ thống....4
Bảng 2.1. Các kỹ thuật đẩy dữ liệu từ máy chủ tới client....6
Bảng 2.2. Chi phí của các mô hình hội nghị với N người tham gia....12
Bảng 2.3. Các khái niệm chính của mediasoup....13
Bảng 3.1. Các tác nhân của hệ thống....15
Bảng 3.2. Yêu cầu chức năng....16
Bảng 3.3. Yêu cầu phi chức năng....16
Bảng 3.4. Tách luồng điều khiển, luồng media và luồng dữ liệu....19
Bảng 3.5. Các bảng dữ liệu theo nhóm nghiệp vụ....21
Bảng 3.6. Quyền ở cấp workspace....23
Bảng 3.7. Quyền ở cấp phiên họp....23
Bảng 3.8. Danh mục sự kiện Socket.IO....24
Bảng 4.1. Cấu trúc mã nguồn....28
Bảng 4.2. Hiện thực trạng thái hiện diện và đang gõ....33
Bảng 5.1. Môi trường thử nghiệm....45
Bảng 5.2. Các bộ kiểm thử tự động....47
Bảng 5.3. Độ trễ tin nhắn tính bằng mili giây (hai lượt mỗi mức)....50
Bảng 5.4. Tài nguyên trong bài kiểm tra tải nhắn tin (giá trị lớn nhất quan sát được)....51
Bảng 5.5. Cấu hình của hai kiến trúc được so sánh....51
Bảng 5.6. Kết quả của hệ thống chuẩn Mesh (trung bình ± độ lệch chuẩn giữa các lượt)....53
Bảng 5.7. Kết quả của SFU có simulcast (trung bình ± độ lệch chuẩn giữa các lượt)....53
Bảng 5.8. Chi phí phía máy chủ của SFU (trung bình giữa các lượt)....57
Bảng 5.9. Các biến thể SFU: simulcast với chính sách mặc định, một bản mã hóa, và ép lớp 0 (trung bình giữa các lượt)....59
Bảng 5.10. So sánh định tính với các nền tảng hiện có....61
Bảng 5.11. Đối chiếu các yêu cầu phi chức năng....62
Bảng 6.1. Mục tiêu ở Chương 1 và mức độ đạt được....65

CHƯƠNG 1. TỔNG QUAN ĐỀ TÀI

1.1 Lý do chọn đề tài

Làm việc từ xa, học trực tuyến và sinh hoạt cộng đồng qua mạng đã trở thành hoạt động thường ngày. Tuy vậy, một nhóm làm việc điển hình vẫn phải dùng song song nhiều nền tảng: một ứng dụng cho trao đổi hằng ngày, một công cụ khác cho họp video và một công cụ nữa để giao việc. Mỗi lần chuyển công cụ là một lần đứt mạch công việc. Ngữ cảnh cuộc thảo luận không đi theo nhóm vào cuộc họp, thông tin "ai đang rảnh" không hiển thị cạnh kênh đang trao đổi, và lịch sử dự án bị phân tán qua nhiều dịch vụ.

Các nền tảng như Discord và Slack cho thấy mô hình tổ chức **không gian làm việc (workspace) chứa các kênh (channel)** giải quyết tốt phần tổ chức của bài toán [1], [2]. Một phòng ban hay một lớp học trở thành một workspace, mỗi chủ đề hay hoạt động là một kênh, và người dùng có thể chuyển giữa kênh văn bản với kênh thoại/video mà không rời ứng dụng. Để xây dựng một nền tảng như vậy cần giải quyết đồng thời hai bài toán kỹ thuật khó: phân phối tin nhắn và trạng thái hiện diện với độ trễ thấp, tin cậy tới nhiều kết nối đồng thời; và tổ chức hội nghị âm thanh/hình ảnh đa điểm ngay trên trình duyệt mà không làm quá tải thiết bị của người tham gia khi phòng đông lên. Đây là hai lõi của kỹ thuật web thời gian thực, nên đề tài phù hợp cho một đồ án tốt nghiệp kết hợp giữa thiết kế phần mềm và đánh giá định lượng.

1.2 Vấn đề cần giải quyết

Đồ án tập trung vào ba vấn đề cụ thể:

1. **Công cụ giao tiếp bị phân mảnh.** Nhắn tin và gọi video nằm ở các ứng dụng khác nhau nên ngữ cảnh công việc bị chia cắt.
2. **Giới hạn của hội nghị Mesh ngang hàng.** Khi mọi người kết nối trực tiếp với nhau, mỗi trình duyệt phải mã hóa và tải lên một luồng riêng cho từng người còn lại. Băng thông tải lên (uplink) và CPU tăng tuyến tính theo số người, nên trình duyệt quá tải ngay khi vài người cùng bật camera.

3. **Đồng bộ trạng thái tức thời.** Trạng thái trực tuyến, đang gõ và tin nhắn mới phải tới đúng người, đúng kênh với độ trễ thấp. Hệ thống còn phải chịu được việc một người mở nhiều tab, mất mạng đột ngột, và máy chủ chạy nhiều tiến trình.

1.3 Mục tiêu

Đồ án đặt ra bốn mục tiêu, được đối chiếu lại ở Chương 6:

- **O1 – Không gian làm việc tích hợp.** Xây dựng một nền tảng web "tất cả trong một" tổ chức theo *Workspace* → *Channel*, cho phép chuyển đổi tự do giữa các workspace và giữa kênh văn bản với kênh thoại/video.
- **O2 – Nhắn tin và trạng thái hiện diện thời gian thực.** Cung cấp nhắn tin thời gian thực theo kênh, không mất và không trùng tin khi gửi lại hay kết nối lại, cùng trạng thái hiện diện đồng bộ (trực tuyến, ngoại tuyến, đang gõ).
- **O3 – Hội nghị truyền hình đa điểm.** Tổ chức hội nghị đa điểm ngay trên trình duyệt, không cài thêm phần mềm, dựa trên SFU để uplink và CPU của client không tăng theo số người tham gia, kèm các chức năng điều hành cuộc họp.
- **O4 – Đánh giá định lượng.** Đo chất lượng truyền (độ trễ, jitter, mất gói), khả năng chịu tải của lõi WebSocket, và so sánh SFU với Mesh trong cùng điều kiện.

1.4 Phạm vi

1.4.1 Đối tượng sử dụng

Nhóm dự án làm việc phân tán, tổ chức giáo dục ứng dụng học trực tuyến và các cộng đồng trên mạng.

1.4.2 Trong phạm vi

- Ứng dụng web đơn trang chạy trên các trình duyệt hiện đại (Chromium, Firefox, Safari).

- Mô hình phân cấp workspace và kênh, phân quyền theo vai trò ở cấp workspace (owner/admin/member) và ở cấp phiên họp (host/co-host/member).
- Kênh văn bản: tin nhắn, trả lời theo luồng (thread), nhắc tên, cảm xúc, ghim, lưu, tệp đính kèm và tìm kiếm.
- Kênh thoại/video: tham gia và rời phòng, bật/tắt micro và camera, chia sẻ màn hình, lưới video động và điều hành phòng (phòng chờ, khóa phòng, cho vào, mời ra, tắt micro người khác, giơ tay, kết thúc).
- Theo dõi chất lượng: thu thập và xuất RTT, jitter, tỉ lệ mất gói và bitrate của cuộc gọi.

Hệ thống còn có một số chức năng hỗ trợ: bảng công việc Kanban, đăng nhập bằng Google và hồ sơ cá nhân. Các chức năng này được trình bày ở chương hiện thực nhưng không phải trọng tâm đánh giá.

1.4.3 Ngoài phạm vi

Ứng dụng di động gốc, ghi hình cuộc họp, mã hóa đầu-cuối xuyên qua SFU và triển khai đa vùng nằm ngoài phạm vi đồ án.

1.5 Phương pháp thực hiện

Đồ án được thực hiện qua bốn bước:

1. **Nghiên cứu lý thuyết** về giao thức WebSocket, bộ giao thức WebRTC (SDP, ICE/STUN/TURN, DTLS-SRTP, RTP/RTCP) và các mô hình hội nghị Mesh, MCU, SFU.
2. **Phân tích và thiết kế**: yêu cầu chức năng và phi chức năng, sơ đồ khối hệ thống với ba khối chính (client, máy chủ ứng dụng/signaling, máy chủ media SFU), cơ sở dữ liệu và mô hình phân quyền.
3. **Hiện thực**: máy chủ Node.js + Socket.IO + PostgreSQL, SFU mediasoup với nhóm worker, và client đơn trang React/TypeScript có lưới video động.
4. **Thực nghiệm và đánh giá**: kiểm thử tự động, kiểm tra tải WebSocket với 50/100/200 kết nối, và một phòng thí nghiệm media trong đó mỗi người

tham gia là một trình duyệt Chromium headless thật với camera giả. Phòng thí nghiệm đo uplink, downlink, CPU, RTT, jitter và tỉ lệ mất gói cho cả hệ thống chuẩn Mesh lẫn SFU.

1.6 Công nghệ sử dụng

Bảng 1.1 liệt kê các công nghệ chính của từng tầng. Client được xây dựng bằng React [3], máy chủ dùng Node.js và Socket.IO [4], còn máy chủ media dùng mediasoup [5].

Bảng 1.1. Công nghệ sử dụng trong hệ thống

Tầng	Công nghệ
Giao diện	React 19, TypeScript 5, Vite 6, mediasoup-client 3, socket.io-client 4
Máy chủ ứng dụng	Node.js 22, TypeScript, Socket.IO 4, JWT (jose), băm mật khẩu scrypt
Máy chủ media	mediasoup 3 (worker C++), simulcast VP8, Opus
Dữ liệu	PostgreSQL 17 (dữ liệu, adapter Socket.IO, presence có TTL); lưu tệp trong cơ sở dữ liệu hoặc S3
Hạ tầng	Docker Compose, Nginx (HTTPS, reverse proxy), coturn (STUN/TURN)
Kiểm thử	node:test, kịch bản tải Socket.IO, phòng thí nghiệm media (Chromium + Chrome DevTools Protocol)

1.7 Bố cục báo cáo

Phần còn lại của báo cáo được tổ chức như sau:

- **Chương 2** tóm tắt cơ sở lý thuyết cần cho đồ án: giao tiếp web thời gian thực, WebRTC và các mô hình hội nghị đa điểm.
- **Chương 3** trình bày yêu cầu, sơ đồ khối hệ thống và phân tích từng khối, cơ sở dữ liệu, phân quyền và các giao thức thời gian thực.
- **Chương 4** mô tả việc triển khai hệ thống qua các sơ đồ giải thuật và những đoạn mã then chốt.
- **Chương 5** trình bày thực nghiệm, đánh giá hệ thống so với hệ thống chuẩn Mesh và so với các yêu cầu đặt ra.

- **Chương 6** kết luận, đối chiếu lại các mục tiêu của chương này và đề xuất hướng phát triển.

CHƯƠNG 2. CƠ SỞ LÝ THUYẾT

Chương này chỉ tóm tắt phần lý thuyết mà đồ án trực tiếp sử dụng: giao tiếp thời gian thực trên web cho nhắn tin và trạng thái hiện diện, WebRTC cho truyền thông đa phương tiện, và ba mô hình hội nghị đa điểm. Phần so sánh cuối chương là cơ sở cho lựa chọn kiến trúc ở Chương 3 và cho thực nghiệm ở Chương 5.

2.1 Giao tiếp thời gian thực trên web

2.1.1 Từ mô hình yêu cầu – phản hồi đến WebSocket

HTTP là giao thức yêu cầu – phản hồi: máy chủ chỉ trả dữ liệu khi trình duyệt hỏi. Để hiển thị tin nhắn mới hay trạng thái "đang gõ" ngay khi nó xảy ra, các kỹ thuật cũ phải giả lập luồng đẩy từ máy chủ, và mỗi kỹ thuật đều có hạn chế riêng (Bảng 2.1).

Bảng 2.1. Các kỹ thuật đẩy dữ liệu từ máy chủ tới client

Kỹ thuật	Cách làm	Hạn chế
Short polling	Client hỏi định kỳ, ví dụ mỗi 2 giây	Trễ tối đa bằng một chu kỳ hỏi; nhiều yêu cầu rỗng
Long polling	Máy chủ giữ yêu cầu tới khi có dữ liệu	Mỗi sự kiện tốn một vòng HTTP mới kèm đầy đủ header
Server-Sent Events	Luồng HTTP một chiều từ máy chủ tới client	Không có chiều client → máy chủ trên cùng kết nối
WebSocket	Một kết nối TCP hai chiều, tồn tại lâu dài	Phải tự quản lý trạng thái kết nối, kết nối lại và mở rộng ngang

WebSocket [6] bắt đầu bằng một yêu cầu `Upgrade` của HTTP/1.1. Client gửi một khóa ngẫu nhiên `Sec-WebSocket-Key`; máy chủ chứng minh mình hiểu giao thức bằng cách trả về giá trị băm SHA-1 (mã hóa base64) của khóa này ghép với một GUID cố định, kèm mã `101 Switching Protocols`. Sau bắt tay, kết nối TCP không còn chạy HTTP mà truyền các khung (frame) WebSocket. Mỗi khung có phần đầu 2–14 byte (bit FIN, opcode, bit mask và độ dài tải); khung từ client phải được che bằng khóa 32 bit để bảo vệ các proxy trung gian, còn các khung ping/pong dùng để

phát hiện kết nối "chết". Vì header chỉ gửi một lần, WebSocket rất phù hợp cho chat, typing và presence, vốn đều cần trao đổi hai chiều với độ trễ thấp. Vì vậy hệ thống dùng WebSocket cho mọi tương tác thời gian thực.

2.1.2 Socket.IO

Socket.IO [4] là thư viện xây dựng trên WebSocket, bổ sung những gì một ứng dụng cần:

- **Sự kiện có tên và xác nhận (acknowledgement).** Máy chủ có thể trả kết quả cho đúng yêu cầu đã phát ra sự kiện, thông qua một hàm callback truyền vào `socket.emit()`. Hệ thống dùng acknowledgement cho mọi thao tác ghi (gửi tin, signaling WebRTC) để client biết chắc thao tác đã thành công hay chưa.
- **Phòng (room).** Máy chủ gom các socket vào phòng, ví dụ theo người dùng (`user:<id>`) hay theo cuộc họp (`meeting:<id>`), rồi phát sự kiện tới cả phòng. Đây là cơ chế đưa tin nhắn tới đúng người nhận.
- **Middleware.** Mỗi kết nối phải mang access token, và middleware có thể kiểm tra mọi sự kiện trước khi tới bộ xử lý.
- **Tự kết nối lại và adapter.** Khi chạy nhiều tiến trình máy chủ, adapter chuyển tiếp lời gọi `io.to(room).emit(...)` sang các tiến trình khác, nhờ đó client nối vào tiến trình A vẫn nhận được sự kiện sinh ra ở tiến trình B.

2.1.3 Đồng bộ trạng thái hiện diện

Presence trả lời câu hỏi "ai đang trực tuyến" trên một tập kết nối luôn biến động, với ba khó khăn. Thứ nhất, một người có thể có nhiều kết nối (nhiều tab, nhiều thiết bị), nên người đó chỉ ngoại tuyến khi kết nối cuối cùng đóng. Thứ hai, kết nối có thể chết mà không báo trước (mất mạng, tiến trình máy chủ bị dừng), nên mỗi bản ghi presence cần một thời gian sống (TTL) được gia hạn bằng heartbeat. Thứ ba, thay đổi trạng thái chỉ nên gửi tới những người cùng workspace với người đó, để số thông điệp tỉ lệ với số người liên quan chứ không phải với toàn hệ thống. Trạng thái đang

gõ là ngắn hạn: client gửi `typing: true` khi bắt đầu gõ và `false` sau vài giây không gõ; máy chủ chỉ chuyển tiếp mà không lưu lại.

2.2 Công nghệ WebRTC

WebRTC [7] là tập API trình duyệt và giao thức IETF cho phép truyền âm thanh, hình ảnh và dữ liệu giữa các điểm cuối mà không cần cài plug-in. Các API chính gồm `getUserMedia()`/`getDisplayMedia()` để thu camera, micro và màn hình; `RTCPeerConnection` để thiết lập phiên; `RTCRtpSender`/`RTCRtpReceiver` để cấu hình mã hóa; và `getStats()` để lấy thống kê theo chuẩn [8], đây cũng là nguồn số liệu QoS ở Chương 5. WebRTC chủ động không định nghĩa signaling: ứng dụng phải tự trao đổi mô tả phiên. Trong hệ thống này, signaling đi qua Socket.IO.

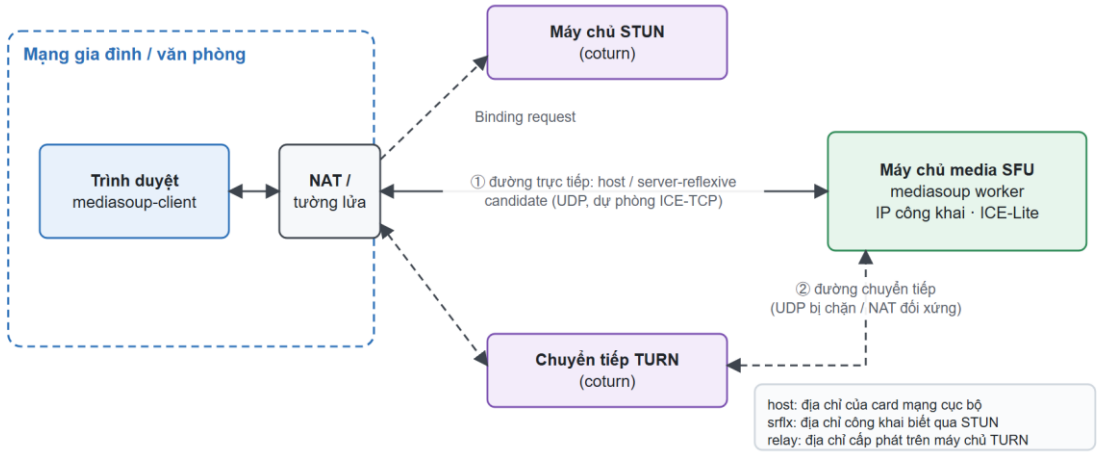
2.2.1 Thương lượng phiên bằng SDP

Một phiên WebRTC được mô tả bằng SDP [9] và thương lượng theo mô hình offer/answer [10] như quy định trong JSEP [11]. Bản mô tả liệt kê một m-section cho mỗi track, các codec và payload type tương ứng (`a=rtpmap`), thông tin xác thực và candidate của ICE, vân tay chứng chỉ DTLS, cùng các tùy chọn như `a=group:BUNDLE` (mọi track dùng chung một transport), `a=rtcp-mux` (RTP và RTCP dùng chung cổng) và `a=simulcast` (nhiều bản mã hóa của cùng một track). Với SFU mediasoup, trình duyệt không thương lượng với một trình duyệt khác: thư viện `mediasoup-client` tự sinh SDP cục bộ từ các tham số mà máy chủ gửi qua signaling (tham số ICE, candidate, tham số DTLS và tham số RTP), còn máy chủ không bao giờ phải phân tích văn bản SDP.

2.2.2 Vượt NAT: ICE, STUN và TURN

Phần lớn thiết bị nằm sau NAT và không có địa chỉ công khai. ICE [12] giải quyết vấn đề này qua ba bước (Hình 2.1). Đầu tiên, mỗi điểm cuối thu thập **candidate**: địa chỉ *host* của chính card mạng, địa chỉ *server-reflexive* biết được nhờ hỏi máy chủ STUN [13], và địa chỉ *relay* được cấp phát trên máy chủ TURN [14]. Tiếp theo, hai bên ghép candidate thành từng cặp và kiểm tra kết nối (gửi STUN Binding request)

theo thứ tự ưu tiên; candidate host có ưu tiên cao nhất và relay thấp nhất, nên ICE luôn ưu tiên đường trực tiếp. Cuối cùng, bên điều khiển đề cử một cặp đã thành công và media đi trên cặp đó. TURN chuyển tiếp toàn bộ media nên tốn băng thông máy chủ, chỉ được dùng khi không thể đi trực tiếp (NAT đối xứng, UDP bị chặn).



Hình 2.1. Các loại candidate ICE và đường kết nối giữa trình duyệt và SFU

Máy chủ media có địa chỉ công khai cố định như mediasoup chạy ở chế độ **ICE-Lite**: máy chủ chỉ công bố host candidate của mình và trả lời các gói kiểm tra, còn trình duyệt điều khiển quá trình. Máy chủ lắng nghe cả UDP và TCP, nên media vẫn đi được qua ICE-TCP khi UDP bị chặn. Để không phải phát một mật khẩu TURN cố định, hệ thống dùng thông tin xác thực TURN có thời hạn [15]: tên người dùng là `<thời điểm hết hạn>:<userId>` và mật khẩu là `base64(HMAC-SHA1(secret, username))`; máy chủ TURN (coturn) kiểm tra lại bằng cùng secret dùng chung.

2.2.3 Bảo mật: DTLS-SRTP

WebRTC bắt buộc mã hóa. Sau khi ICE tìm được đường đi, hai bên chạy bắt tay DTLS (TLS trên UDP) và so chứng chỉ của bên kia với vân tay đã nhận qua signaling; vì signaling chạy trên WebSocket bảo mật nên vân tay không thể bị giả mạo. Khóa sinh ra từ DTLS [16] được SRTP [17] dùng để mã hóa và xác thực từng gói RTP và RTCP. Với kiến trúc SFU, việc mã hóa này là theo từng chặng (client ↔ SFU): SFU giải mã gói để đọc header RTP rồi mã hóa lại cho từng người nhận.

2.2.4 RTP, RTCP và các đại lượng QoS

RTP [18] mang dữ liệu media. Header cố định 12 byte của nó chứa payload type, **số thứ tự** 16 bit (để phát hiện mất và đảo thứ tự gói), **nhãn thời gian** 32 bit theo đồng hồ của codec và **SSRC** 32 bit định danh luồng. RTCP là kênh điều khiển đi kèm: các báo cáo Sender Report và Receiver Report mang số gói, số gói mất tích lũy, số thứ tự lớn nhất và jitter; các gói phản hồi mang NACK (yêu cầu gửi lại), PLI (yêu cầu khung khóa mới) và phản hồi phục vụ điều khiển tắc nghẽn. Các đại lượng QoS dùng trong đồ án gồm:

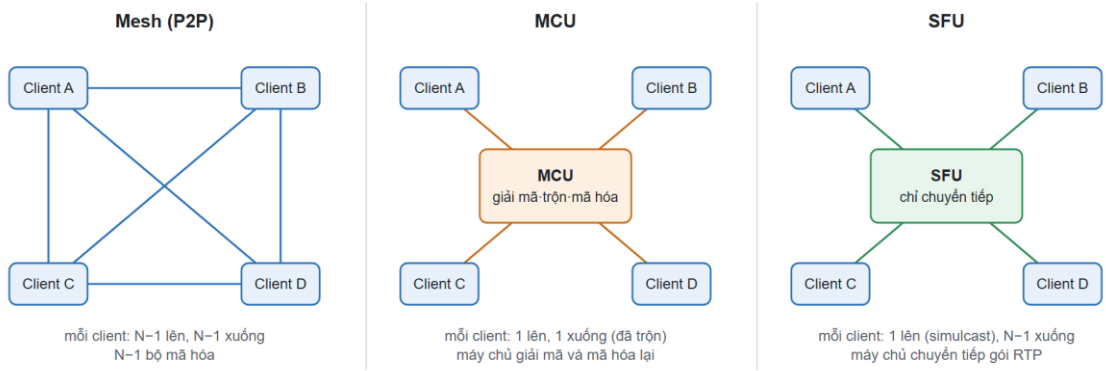
- **Jitter** là độ biến thiên thời gian truyền của gói. Với $D(i-1,i) = (R_i - R_{i-1}) - (S_i - S_{i-1})$, trong đó R là thời điểm nhận và S là nhãn thời gian RTP, ước lượng được làm trơn theo $J_i = J_{i-1} + (|D(i-1,i)| - J_{i-1})/16$.
- **Thời gian khứ hồi (RTT)**, đo từ báo cáo RTCP hoặc, trong trình duyệt, từ các gói kiểm tra STUN trên cặp candidate ICE đang được chọn.
- **Tỉ lệ mất gói** trong một cửa sổ đo: $\Delta lost / (\Delta received + \Delta lost)$.
- **Độ trễ jitter buffer** là thời gian trung bình một khung hình nằm trong bộ đệm nhận trước khi được giải mã, thành phần chính của độ trễ đầu-cuối mà người dùng cảm nhận.

2.2.5 Điều khiển tắc nghẽn, simulcast và codec

Băng thông mạng không cố định nên bên gửi phải ước lượng liên tục. Chrome dùng Google Congestion Control (GCC) [19], coi độ trễ hàng đợi tăng là dấu hiệu sắp tắc nghẽn để giảm bitrate, độ phân giải hoặc tốc độ khung; trình duyệt báo lý do trong `qualityLimitationReason` (`bandwidth` hoặc `cpu`). Để phục vụ nhiều người nhận có băng thông khác nhau, bên gửi có thể dùng **simulcast**: mã hóa cùng một video thành nhiều bản ở các độ phân giải khác nhau, mỗi bản là một luồng RTP riêng, để SFU chuyển tiếp bản phù hợp cho từng người nhận; cái giá là bên gửi tốn thêm bộ mã hóa và uplink. Hệ thống dùng simulcast VP8 [20] ba lớp *q/h/f* (một phần tư, một nửa và toàn bộ độ phân giải, tối đa 150/400/1200 kbps) và Opus [21] cho âm thanh.

2.3 Các mô hình hội nghị đa điểm

Xét một phòng có N người tham gia, mỗi người gửi một luồng video có bitrate b. Hình 2.2 minh họa ba mô hình cổ điển.



Hình 2.2. Ba mô hình Mesh, MCU và SFU với một phòng bốn người

2.3.1 Mesh

Mỗi cặp người tham gia có một kết nối ngang hàng riêng, tổng cộng N(N−1)/2 kết nối. Mỗi client phải mã hóa video N−1 lần (mỗi kết nối một bộ mã hóa vì mỗi kết nối có điều khiển tắc nghẽn riêng), tải lên N−1 luồng và giải mã N−1 luồng. Mô hình này không cần máy chủ media và có độ trễ thấp nhất, nhưng uplink và CPU của mọi client tăng tuyến tính theo N. Vì uplink thường là phần yếu nhất của mạng gia đình và mạng di động, Mesh chỉ dùng được cho phòng rất nhỏ.

2.3.2 MCU

Mỗi client gửi một luồng tới Multipoint Control Unit; MCU giải mã tất cả, trộn thành một khung hình và một luồng âm thanh tổng hợp, rồi mã hóa lại cho từng người. Client chỉ gửi và nhận một luồng, nhưng máy chủ tốn rất nhiều CPU cho giải mã, trộn và mã hóa, làm tăng độ trễ xử lý, áp đặt một bố cục cố định và làm giảm chất lượng do mã hóa lại.

2.3.3 SFU

Mỗi client gửi một luồng (hoặc một nhóm simulcast) tới Selective Forwarding Unit. SFU không giải mã media: nó chỉ đọc header RTP và chuyển tiếp mỗi gói tới

đúng những người nhận cần nó, đồng thời xử lý phản hồi RTCP cho từng người nhận. Uplink của client không phụ thuộc N, CPU máy chủ thấp vì chỉ làm việc ở mức gói tin, và mỗi người nhận có thể nhận một lớp chất lượng khác nhau. Downlink của client vẫn tăng theo N−1, nhưng có thể giảm bằng cách chọn lớp thấp hơn hoặc tạm dừng các luồng không hiển thị [22].

2.3.4 So sánh định lượng

Bảng 2.2 so sánh chi phí của các mô hình đối với mỗi client và đối với máy chủ.

Bảng 2.2. Chi phí của các mô hình hội nghị với N người tham gia

Đại lượng (mỗi client)	**Mesh**	**MCU**	**SFU**	**SFU + simulcast**
Số bộ mã hóa video	N−1	1	1	3 (cố định)
Uplink	(N−1)·b	b	b	b_q + b_h + b_f
Downlink	(N−1)·b	b_mix	(N−1)·b	tổng các lớp được chọn
Số luồng giải mã	N−1	1	N−1	N−1 hoặc số ô hiển thị
CPU máy chủ	không có	rất cao	thấp	thấp
Độ trễ thêm	không có	giải mã + trộn + mã hóa	một chặng chuyển tiếp	một chặng chuyển tiếp

Với N = 6 và b = 1,2 Mbps, mỗi client Mesh phải tải lên 5 × 1,2 = 6 Mbps với năm bộ mã hóa, trong khi client SFU có simulcast chỉ tải lên tối đa 0,15 + 0,4 + 1,2 ≈ 1,75 Mbps với ba bộ mã hóa, bất kể N. Các nghiên cứu về SFU mã nguồn mở [23] cũng ghi nhận cùng xu hướng định tính này. Vì vậy đồ án chọn **SFU**: uplink và CPU của client không đổi khi N tăng, máy chủ không phải giải mã media, và simulcast kết hợp với chọn lớp theo nhu cầu giúp kiểm soát cả downlink. Chương 5 kiểm chứng lựa chọn này bằng thực nghiệm so với một hiện thực Mesh trên cùng phần cứng.

2.4 Thư viện SFU mediasoup

mediasoup [5] là thư viện SFU cho Node.js; phần lõi xử lý media viết bằng C++ và chạy trong các tiến trình worker riêng, còn API điều khiển viết bằng JavaScript/TypeScript. Các khái niệm chính được liệt kê ở Bảng 2.3.

Bảng 2.3. Các khái niệm chính của mediasoup

Khái niệm	Vai trò
Worker	Tiến trình C++ dùng một lõi CPU; ứng dụng tạo một nhóm worker
Router	Một "phòng" media với tập codec chung; chuyển gói từ producer tới consumer
WebRtcTransport	Điểm cuối ICE-Lite/DTLS/SRTP cho một client (thường có một transport gửi và một transport nhận)
Producer	Một track do client gửi lên (micro, camera hoặc màn hình)
Consumer	Một track SFU gửi xuống một client; có thể tạm dừng, tiếp tục và chuyển giữa các lớp simulcast

Luồng signaling tối thiểu gồm: lấy RTP capabilities của router, tạo hai transport, kết nối chúng bằng tham số DTLS của client, produce các track cục bộ và consume các track của người khác. Consumer được tạo ở trạng thái **tạm dừng** và chỉ tiếp tục khi client đã sẵn sàng, để không mất khung khóa đầu tiên. mediasoup còn gửi các gói thăm dò trên một luồng riêng (SSRC 1234) để tìm băng thông còn trống trước khi nâng lớp cho consumer; các gói này phải được loại khỏi chỉ số chất lượng phía client (mục 5.4.2).

2.5 Kiểm soát truy cập theo vai trò

Kiểm soát truy cập theo vai trò (RBAC) [24] gán quyền cho vai trò thay vì cho từng người dùng. Hệ thống dùng hai cấp vai trò. Ở **cấp workspace**, *owner* duy nhất có toàn quyền và có thể chuyển giao quyền sở hữu, *admin* quản lý thành viên, kênh và lời mời, còn *member* dùng các kênh mình được phép thấy; kênh riêng tư có thêm danh sách thành viên kênh. Ở **cấp phiên họp**, *host* và *co-host* điều hành phòng (phòng chờ, khóa phòng, cho vào, mời ra, tắt micro), và chỉ host mới được chỉ định co-host

hoặc kết thúc cuộc họp. Mọi sự kiện socket có ghi dữ liệu và mọi lời gọi signaling media đều được kiểm tra theo vai trò hiện tại trước khi tới bộ xử lý.

2.6 Tóm tắt chương

WebSocket và Socket.IO cung cấp kênh hai chiều có xác nhận, có phòng và mở rộng ngang được nhờ adapter; đây là nền tảng cho chat, typing, presence và signaling. WebRTC đảm nhận việc thu, mã hóa, truyền media an toàn và cung cấp thống kê chuẩn; ICE/STUN/TURN giải quyết NAT, còn RTP/RTCP mang media và phản hồi chất lượng. Trong ba mô hình hội nghị, SFU cân bằng tốt nhất giữa tải client, tải máy chủ và độ trễ, và với simulcast có thể kiểm soát cả uplink lẫn downlink. Các kết luận này định hướng thiết kế ở Chương 3.

CHƯƠNG 3. SƠ ĐỒ KHỐI VÀ THIẾT KẾ HỆ THỐNG

Chương này đi từ yêu cầu, trình bày sơ đồ khối của toàn hệ thống, rồi phân tích từng khối: khối gồm những thành phần nào, đảm nhận chức năng gì và phối hợp với các khối khác ra sao. Các mục cuối chương mô tả cơ sở dữ liệu, mô hình phân quyền và các giao thức thời gian thực kết nối các khối với nhau.

3.1 Phân tích yêu cầu

3.1.1 Tác nhân

Bảng 3.1 liệt kê các tác nhân của hệ thống. Vai trò trong workspace và vai trò trong phiên họp là độc lập: cùng một người có thể là thành viên của workspace và là host của một cuộc họp trong một kênh thuộc workspace đó.

Bảng 3.1. Các tác nhân của hệ thống

Tác nhân	Mô tả
Khách	Chưa đăng nhập; xem trang giới thiệu, đăng ký, đăng nhập và mở liên kết mời
Thành viên	Thuộc một hoặc nhiều workspace; nhắn tin và tham gia kênh thoại/video
Quản trị viên (admin)	Thành viên được giao quản lý kênh, thành viên và lời mời của workspace
Chủ sở hữu (owner)	Người tạo workspace; toàn quyền, đổi vai trò, chuyển giao quyền sở hữu, lưu trữ
Host / co-host	Vai trò trong một phiên họp; điều hành phòng

3.1.2 Yêu cầu chức năng

Bảng 3.2 liệt kê các yêu cầu chức năng theo nhóm. F1–F7 là các chức năng dành cho người dùng; F8 phục vụ việc đánh giá ở Chương 5.

Bảng 3.2. Yêu cầu chức năng

Mã	Nhóm	Yêu cầu
F1	Tài khoản	Đăng ký có xác thực OTP qua e-mail, đăng nhập bằng mật khẩu hoặc Google, làm mới phiên, đặt lại mật khẩu
F2	Workspace	Tạo workspace; mời thành viên qua liên kết hoặc e-mail; vai trò owner/admin/member; lưu trữ và khôi phục
F3	Kênh	Kênh văn bản, thoại và video; kênh riêng tư có danh sách thành viên; sắp xếp kênh
F4	Nhắn tin	Gửi, sửa, xóa tin; trả lời theo luồng, nhắc tên, cảm xúc, ghim, lưu, tệp đính kèm, tìm kiếm
F5	Trạng thái	Trạng thái trực tuyến/ngoại tuyến, đang gõ và số tin chưa đọc theo thời gian thực
F6	Hội nghị	Tham gia và rời kênh video; bật/tắt micro, camera; chia sẻ màn hình; lưới video động
F7	Điều hành họp	Phòng chờ, khóa phòng, cho vào, mời ra, tắt micro người khác, giơ tay, kết thúc cuộc họp
F8	Đo kiểm	Hiển thị và xuất số liệu QoS của cuộc gọi (RTT, jitter, mất gói, bitrate)

3.1.3 Yêu cầu phi chức năng

Bảng 3.3 nêu các yêu cầu phi chức năng và cách kiểm chứng từng yêu cầu ở Chương 5.

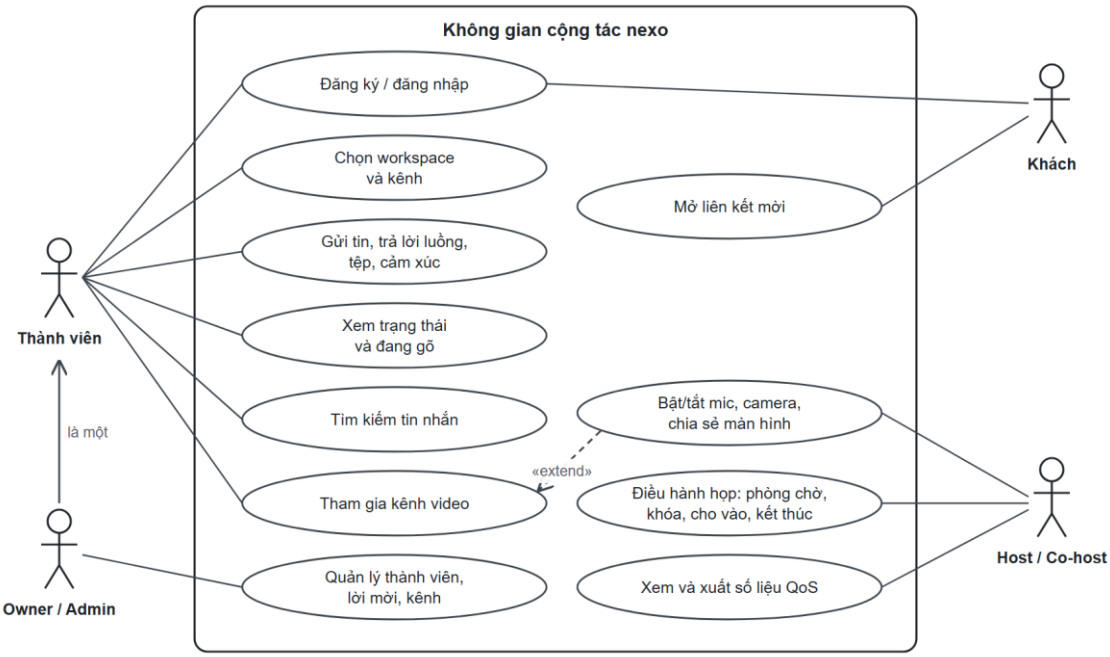
Bảng 3.3. Yêu cầu phi chức năng

Nhóm	Yêu cầu	Cách kiểm chứng
Độ trễ	Tin nhắn tới người nhận dưới 100 ms (phân vị 95) trong mạng nội bộ	Kiểm tra tải với 50/100/200 kết nối (mục 5.3)
Khả năng mở rộng media	Uplink và CPU của client không tăng theo số người tham gia	Thực nghiệm Mesh/SFU (mục 5.4)
Chất lượng media	Mất gói ≈ 0, jitter < 30 ms, RTT < 150 ms khi mạng tốt	Số liệu `getStats()` (mục 5.4)
Độ tin cậy	Không mất, không trùng tin khi mất xác nhận hoặc kết nối lại; tự phục hồi ICE	Kiểm thử tích hợp (mục 5.2)
Bảo mật	Băm mật khẩu, token ngắn hạn, kiểm quyền từng sự kiện, DTLS-SRTP	Kiểm thử phân quyền (mục 5.2)

Nhóm	Yêu cầu	Cách kiểm chứng
Mở rộng ngang	Chạy nhiều tiến trình API; phân bổ cuộc họp theo worker/node	Kiểm thử tích hợp hai tiến trình (mục 5.2)

3.1.4 Ca sử dụng chính

Hình 3.1 trình bày các ca sử dụng chính theo từng tác nhân. Thành viên làm việc trong workspace và kênh; owner và admin quản lý workspace; host và co-host điều hành cuộc họp.

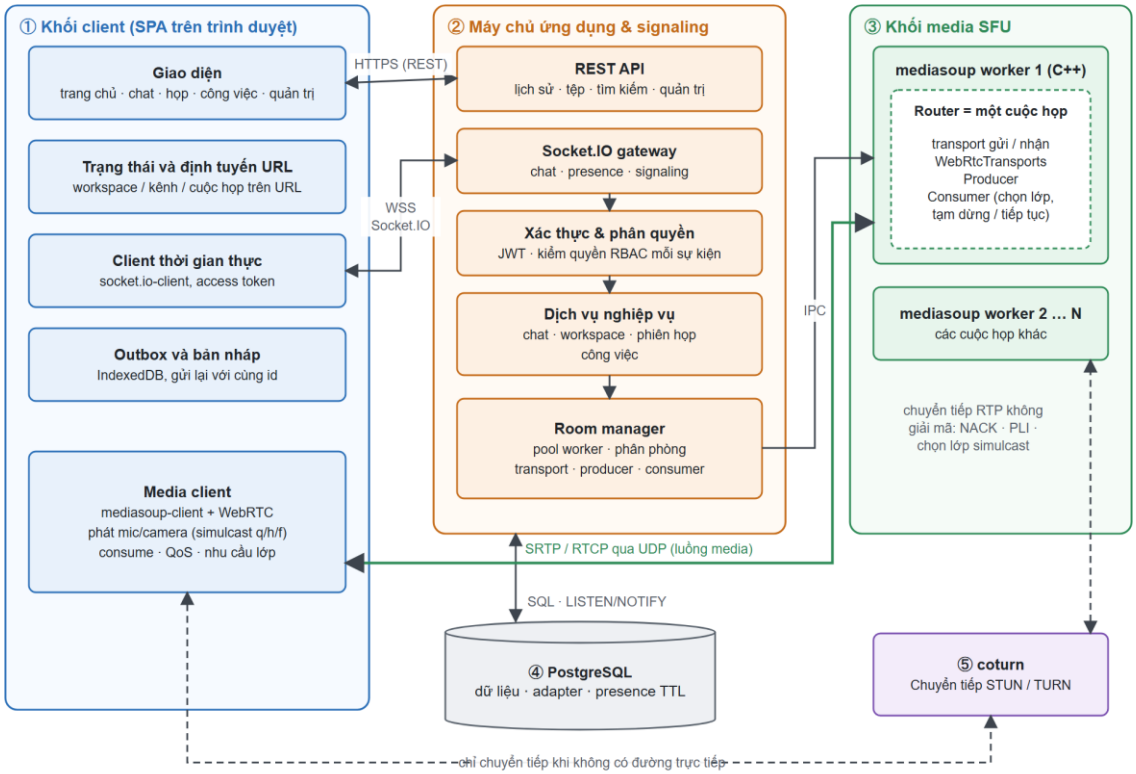


Hình 3.1. Các ca sử dụng chính của hệ thống

3.2 Sơ đồ khối hệ thống

3.2.1 Tổng quan

Hình 3.2 là sơ đồ khối của hệ thống, gồm ba khối chính (client, máy chủ ứng dụng và signaling, máy chủ media SFU) và hai khối hỗ trợ (khối dữ liệu và khối vượt NAT).



Hình 3.2. Sơ đồ khối hệ thống

- **Khối client** là ứng dụng đơn trang React chạy trong trình duyệt. Khối này mở một kết nối Socket.IO cho nhắn tin và trạng thái hiện diện, dùng REST cho dữ liệu tải theo yêu cầu, và mở hai transport WebRTC (gửi và nhận) khi người dùng vào kênh video.
- **Khối máy chủ ứng dụng và signaling** là một tiến trình Node.js cung cấp REST API và cổng Socket.IO. Mọi yêu cầu và mọi sự kiện đều qua bước xác thực và phân quyền trước khi tới các dịch vụ chat, họp hay công việc. Room manager trong khối này điều khiển các worker media.
- **Khối media SFU** là nhóm worker C++ của mediasoup. Mỗi cuộc họp là một router đặt trên một worker; media đi thẳng giữa trình duyệt và worker.
- **Khối dữ liệu** là PostgreSQL. Ngoài dữ liệu nghiệp vụ, khối này lưu trạng thái hiện diện có TTL và chuyển tiếp sự kiện Socket.IO giữa các tiến trình ứng dụng.

- **Khối vượt NAT** là máy chủ coturn cung cấp STUN/TURN, dùng khi không thể có đường UDP trực tiếp giữa trình duyệt và SFU.

3.2.2 Tách luồng điều khiển và luồng dữ liệu

Quyết định thiết kế quan trọng nhất là tách luồng điều khiển khỏi luồng media (Bảng 3.4). Tin nhắn, trạng thái hiện diện và signaling là các thông điệp nhỏ, đi qua tiến trình Node.js; còn gói âm thanh và hình ảnh có kích thước lớn, không bao giờ đi qua Node.js mà truyền thẳng giữa trình duyệt và worker C++. Nhờ vậy tiến trình Node.js chỉ phải xử lý thông điệp nhỏ, và Chương 5 cho thấy CPU của tiến trình này chỉ khoảng 1% một lõi hoặc ít hơn trong khi các worker chuyển tiếp media cho mười người.

Bảng 3.4. Tách luồng điều khiển, luồng media và luồng dữ liệu

Luồng	Giao thức	Đường đi
Điều khiển (tin nhắn, presence, signaling)	WebSocket bảo mật (Socket.IO)	Trình duyệt ↔ API Node.js
Media	SRTP/SRTCP trên UDP (dự phòng ICE-TCP, TURN)	Trình duyệt ↔ worker mediasoup
Dữ liệu bền vững	SQL	API Node.js ↔ PostgreSQL

3.3 Phân tích từng khối

3.3.1 Khối client

Khối client gồm năm thành phần:

1. **Giao diện** cho trang chủ, workspace, kênh, chat, cuộc họp, công việc, thông báo, hồ sơ và quản trị. Bố cục gồm thanh điều hướng, danh sách kênh và vùng nội dung; trên điện thoại, các cột thu lại thành ngăn kéo.
2. **Trạng thái ứng dụng và định tuyến.** Workspace, kênh và cuộc họp hiện tại được mã hóa trên URL, nên tải lại trang, bấm quay lại/tiến tới hay chia sẻ liên kết đều khôi phục đúng ngữ cảnh.

3. **Client thời gian thực.** Một kết nối `socket.io-client` duy nhất, xác thực bằng access token ngắn hạn. Thành phần này nhận tin nhắn, trạng thái hiện diện và sự kiện cuộc họp, đồng thời gửi các yêu cầu signaling có acknowledgement.
4. **Outbox và bản nháp bền vững.** Tin chưa được xác nhận và bản nháp được lưu trong IndexedDB, nên vẫn còn sau khi tải lại trang và có thể gửi lại với cùng mã định danh.
5. **Media client.** `mediasoup-client` dựng các transport WebRTC từ tham số do máy chủ gửi, phát micro, camera (ba lớp simulcast) và màn hình, nhận các track từ xa, thu thập số liệu QoS và quyết định lớp cần thiết cho từng ô video đang hiển thị.

3.3.2 Khối máy chủ ứng dụng và signaling

Khối máy chủ chạy trên Node.js, gồm:

1. **Xác thực**: đăng ký có OTP qua e-mail, đăng nhập bằng mật khẩu được băm scrypt có salt [25], đăng nhập Google, access token JWT [26] ngắn hạn (15 phút) và refresh token xoay vòng lưu trong cookie HttpOnly.
2. **REST API** cho dữ liệu tải theo yêu cầu: workspace, lịch sử kênh, tệp, tìm kiếm, thông báo, công việc và quản trị.
3. **Cổng Socket.IO** với hai lớp middleware: kiểm tra token khi kết nối, và giới hạn tần suất cùng kiểm quyền trước mỗi sự kiện.
4. **Các dịch vụ nghiệp vụ**: chat (tin nhắn, luồng trả lời, nhắc tên, thông báo), workspace và lời mời, phiên họp và điều hành, công việc.
5. **Room manager**, quản lý nhóm worker mediasoup, đặt mỗi cuộc họp lên một worker và thay mặt client tạo transport, producer và consumer.

3.3.3 Khối media SFU

Khối SFU là một nhóm worker mediasoup (mặc định hai, tối đa tám). Mỗi cuộc họp là một router nằm trên đúng một worker, được chọn là worker đang giữ ít phòng nhất. Router chứa một transport gửi và một transport nhận cho mỗi người tham

gia, các producer mà người tham gia phát lên và các consumer mang track của người khác xuống. Worker chuyển tiếp gói RTP mà không giải mã, chọn lớp simulcast cho từng consumer, gửi lại gói bị mất khi nhận NACK và yêu cầu khung khóa khi consumer bắt đầu hoặc đổi lớp. Nếu một worker chết, chỉ các cuộc họp trên worker đó bị đóng; người tham gia được yêu cầu vào lại và một worker thay thế được tạo.

3.3.4 Khối dữ liệu

PostgreSQL lưu toàn bộ dữ liệu nghiệp vụ (mục 3.4) và còn đóng hai vai trò hạ tầng. Thứ nhất, adapter PostgreSQL của Socket.IO dùng cơ chế `LISTEN/NOTIFY` [27] để chuyển tiếp sự kiện giữa các tiến trình API. Thứ hai, bảng `socket_presence` lưu một dòng cho mỗi socket còn sống kèm thời điểm hết hạn được gia hạn bằng heartbeat, nhờ đó trạng thái hiện diện nhất quán giữa các tiến trình và các dòng của một tiến trình bị dừng đột ngột sẽ tự hết hạn.

3.3.5 Khối vượt NAT

Máy chủ coturn cung cấp STUN và TURN. API sinh thông tin xác thực TURN có thời hạn cho mỗi transport, nhờ đó client có thể chuyển sang đường chuyển tiếp khi UDP bị chặn hoặc NAT là loại đối xứng. Bản thân SFU công bố địa chỉ công khai (hoặc địa chỉ LAN) và chấp nhận cả UDP lẫn TCP.

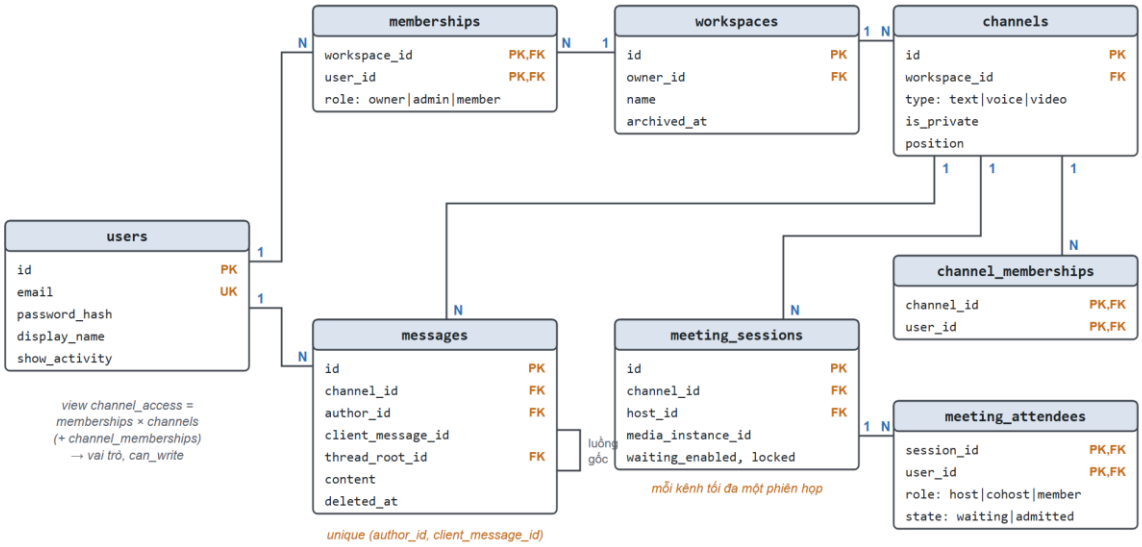
3.4 Thiết kế cơ sở dữ liệu

Cơ sở dữ liệu PostgreSQL gồm 36 bảng và view, được tạo bởi 20 migration có đánh số phiên bản. Bảng 3.5 nhóm các bảng theo nghiệp vụ, còn Hình 3.3 trình bày các thực thể cốt lõi của phần nhắn tin và hội nghị.

Bảng 3.5. Các bảng dữ liệu theo nhóm nghiệp vụ

Nhóm	Bảng chính	Ghi chú
Định danh	`users`, `refresh_sessions`, `email_otps`, `password_resets`	Mật khẩu băm có salt; refresh token và mã OTP chỉ lưu dạng băm
Tổ chức	`workspaces`, `memberships`, `channels`, `channel_memberships`,	Đúng một owner (unique index và constraint trigger); token mời lưu dạng SHA-256

Nhóm	Bảng chính	Ghi chú
	`workspace_invitations`, `audit_events`	
Giao tiếp	`messages`, `message_attachments`, `message_reactions`, `message_mentions`, `message_pins`, `message_bookmarks`, `thread_reads`, `notifications`	Mã tin do client sinh là duy nhất theo tác giả, chống tin trùng
Hội nghị	`meeting_sessions`, `meeting_attendees`, `meeting_connections`, `meeting_schedules`	Mỗi kênh tối đa một phiên họp đang hoạt động (partial unique index)
Vận hành	`socket_presence`, `api_instances`, `request_rate_limits`, `schema_migrations`	TTL presence, lease của tiến trình, giới hạn tần suất
Mở rộng	`tasks` và các bảng liên quan	Bảng công việc Kanban (không phải trọng tâm đồ án)



Hình 3.3. Các thực thể cốt lõi của cơ sở dữ liệu

View `channel_access` ghép `memberships` với `channels` (cùng `channel_memberships` đối với kênh riêng tư) và trả về, cho mỗi cặp người dùng – kênh, vai trò cùng quyền ghi (workspace và kênh chưa bị lưu trữ). Mọi phép kiểm tra quyền đọc/ghi (REST, socket, tìm kiếm, tải tệp và media) đều dùng chung view này, nên chính sách truy cập chỉ được định nghĩa ở một nơi duy nhất.

3.5 Thiết kế phân quyền

Bảng 3.6 và Bảng 3.7 định nghĩa quyền của hai cấp vai trò đã giới thiệu ở mục 2.5.

Bảng 3.6. Quyền ở cấp workspace

Thao tác	Owner	Admin	Member	Ghi chú
Đọc và gửi tin trong kênh công khai	✓	✓	✓	Workspace và kênh chưa bị lưu trữ
Đọc và gửi tin trong kênh riêng tư	✓*	✓*	✓*	*Phải là thành viên kênh, kể cả owne*
Tạo, sửa và sắp xếp kênh	✓	✓	–	
Mời và thu hồi lời mời	✓	✓	–	Token có hạn 7 ngày, chỉ lưu dạng băm
Đổi vai trò, chuyển quyền sở hữu, lưu trữ	✓	–	–	Giao dịch có khóa workspace
Tham gia kênh video	✓	✓	✓	Giới hạn số người, số tab và số nguồn mỗi phiên

Bảng 3.7. Quyền ở cấp phiên họp

Thao tác	Host	Co-host	Member
Bật phòng chờ, khóa phòng	✓	✓	–
Cho vào, mời ra, tắt micro người khác	✓	✓	–
Chỉ định co-host, kết thúc phiên	✓	–	–
Giơ tay, bật/tắt thiết bị của mình	✓	✓	✓

Khi một người dùng bị thu hồi quyền, máy chủ phát sự kiện `access_revoked`, ngắt mọi socket của người đó trên mọi tiến trình và đóng tài nguyên họp của họ. Client sau đó kết nối lại và tải dữ liệu theo quyền mới.

3.6 Thiết kế giao thức thời gian thực

3.6.1 Danh mục sự kiện

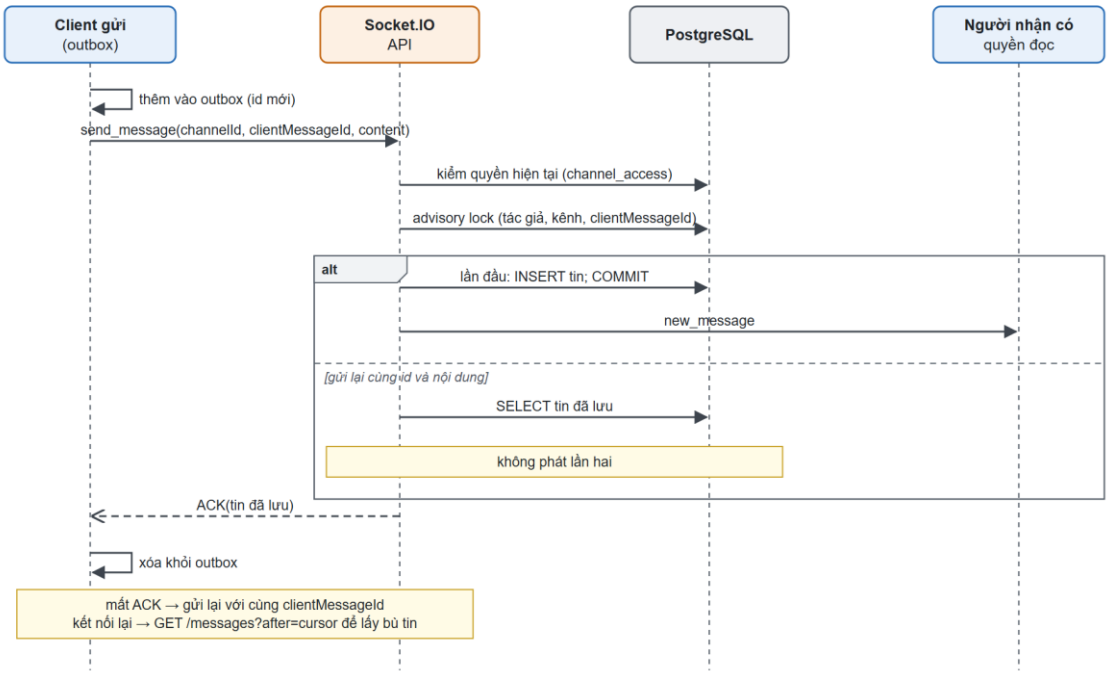
Hợp đồng sự kiện Socket.IO được định nghĩa một lần bằng kiểu TypeScript trong một gói dùng chung cho cả client và máy chủ, nên việc đổi tên sự kiện hay đổi kiểu dữ liệu sẽ bị trình biên dịch báo lỗi ở cả hai phía. Bảng 3.8 tóm tắt các sự kiện.

Bảng 3.8. Danh mục sự kiện Socket.IO

Nhóm	Client → máy chủ (có acknowledgement)	Máy chủ → client
Kênh	`join_channel`, `leave_channel`, `mark_channel_read`	`channel_read`
Tin nhắn	`send_message`, `edit_message`, `delete_message`, `react_message`	`new_message`, `message_changed`
Trạng thái	`typing`	`user_typing`, `user_presence`, `online_users`
Cuộc họp	`join_meeting`, `leave_meeting`, `meeting_state`, `meeting_action`	`meeting_state_changed`, `meeting_closed`, `meeting_mute`
Signaling SFU	`create_webrtc_transport`, `connect_webrtc_transport`, `produce`, `consume`, `resume_consumer`, `pause_consumer`, `consumer_layer`, `close_producer`, `restart_ice`	`new_producer`, `producer_closed`, `peer_left`

3.6.2 Luồng gửi tin nhắn

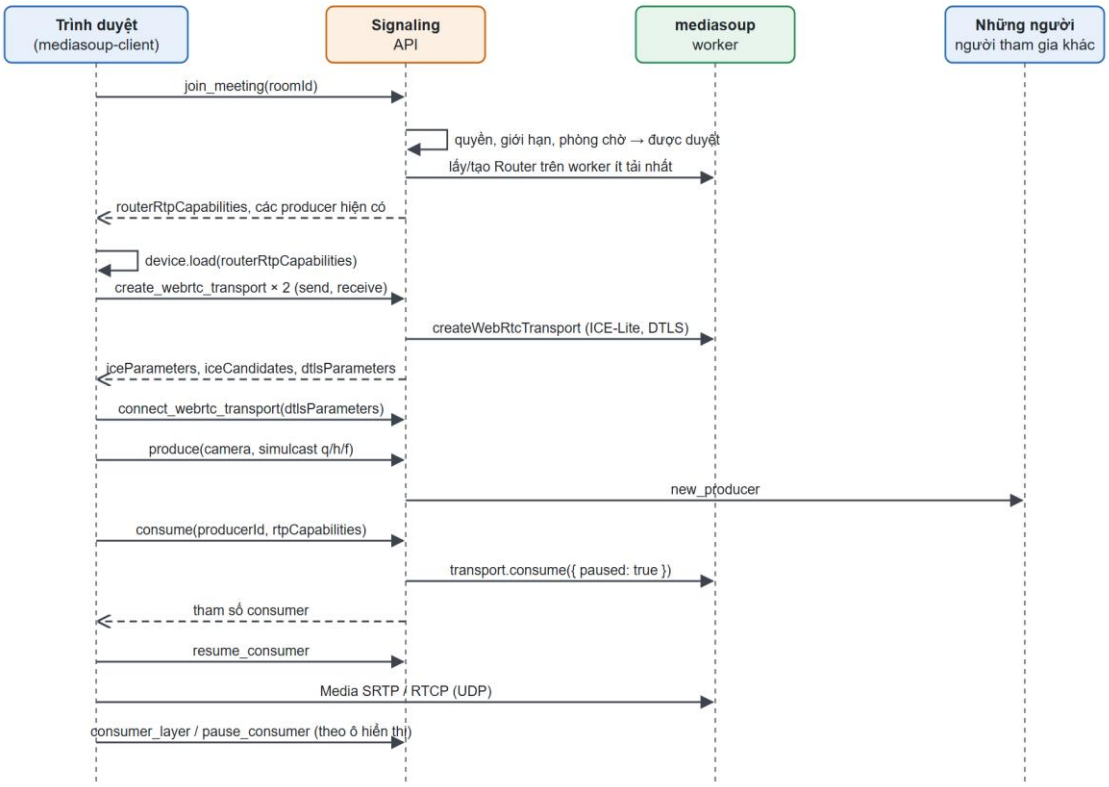
Mục tiêu của giao thức nhắn tin là một tin không bao giờ bị mất và không bao giờ được lưu hai lần, kể cả khi mất acknowledgement hay rớt kết nối. Hình 3.4 trình bày luồng xử lý. Client gán một `clientMessageId` và giữ tin trong outbox cho tới khi nhận được acknowledgement. Máy chủ kiểm tra quyền hiện tại, khóa theo cặp (tác giả, clientMessageId), rồi hoặc chèn tin mới và phát cho người nhận, hoặc, nếu cùng mã định danh đã được lưu với cùng nội dung, trả lại tin đã lưu mà không phát lại. Khi mất acknowledgement, client gửi lại với cùng mã; sau khi kết nối lại, client lấy bù các tin đã lỡ qua REST với con trỏ `after`.



Hình 3.4. Sơ đồ tuần tự gửi tin nhắn tin cậy

3.6.3 Tham gia kênh video

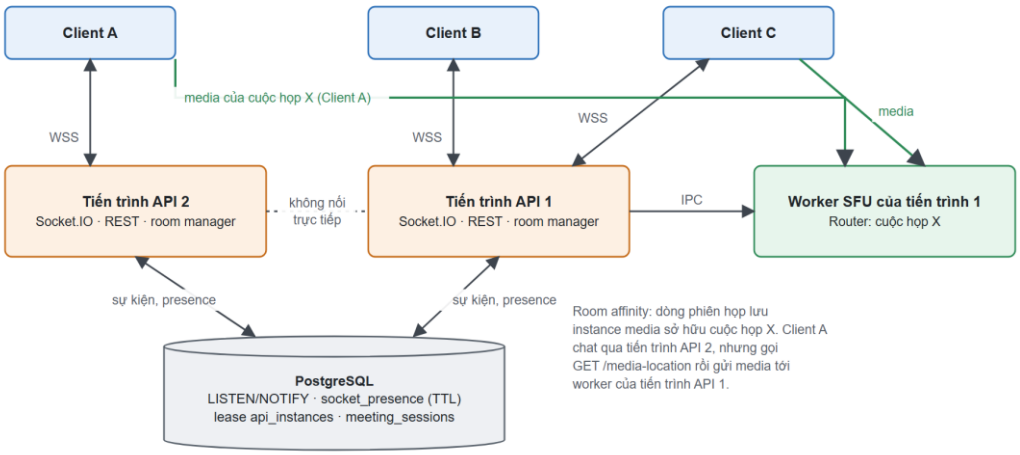
Hình 3.5 trình bày cách một client tham gia kênh video. Máy chủ kiểm tra quyền, giới hạn của phiên và phòng chờ, đặt cuộc họp lên một worker và trả về RTP capabilities của router. Client nạp thiết bị mediasoup, tạo transport gửi và transport nhận, phát các track của mình (camera dưới dạng ba lớp simulcast) và consume các producer của người khác. Consumer được tạo ở trạng thái tạm dừng và chỉ tiếp tục khi client đã sẵn sàng. Sau đó client liên tục báo cho SFU lớp cần thiết của từng ô video, hoặc tạm dừng các ô không hiển thị.



Hình 3.5. Sơ đồ tuần tự tham gia kênh video

3.7 Thiết kế mở rộng ngang

Nhắn tin và media được mở rộng theo hai cách khác nhau, như minh họa ở Hình 3.6. Với nhắn tin và trạng thái hiện diện, nhiều tiến trình API dùng chung PostgreSQL; adapter PostgreSQL chuyển tiếp sự kiện giữa các tiến trình, còn bảng `socket_presence` có TTL và heartbeat cho một cái nhìn nhất quán về việc ai đang trực tuyến. Mỗi tiến trình còn gia hạn một lease trong `api_instances`, để các tiến trình khác phát hiện tiến trình đã chết và dọn các cuộc họp của nó. Media không thể chia sẻ theo cách đó vì transport và router nằm bên trong một tiến trình worker. Do vậy một cuộc họp chỉ nằm trên **một** node và worker (room affinity): dòng phiên họp ghi lại instance media sở hữu cuộc họp, và client hỏi API vị trí media trước khi mở socket media.



Hình 3.6. Mở rộng ngang: nhắn tin dùng chung, media theo room affinity

3.8 Thiết kế giao diện

Giao diện theo bố cục ba cột: thanh điều hướng (trang chủ, nhóm, cuộc họp, công việc, thông báo, cài đặt), danh sách workspace và kênh, và vùng nội dung cho chat hoặc cuộc họp, kèm một panel thông tin có thể đóng mở. Lưới video tính số cột cho ô 16:9 lớn nhất, chia trang (4, 6 hoặc 9 ô mỗi trang tùy bề rộng) và có chế độ tiêu điểm với một ô lớn. Ô nằm ngoài trang hiện tại không được render và luồng video tương ứng bị tạm dừng tại SFU. Trạng thái kết nối luôn hiển thị: tiêu đề kênh cho biết số thành viên đang trực tuyến hoặc trạng thái "đang kết nối lại", còn tin chưa gửi được hiển thị kèm thao tác *gửi lại* và *bỏ*. Ảnh chụp giao diện đã hiện thực được trình bày ở mục 4.5.

3.9 Tóm tắt chương

Hệ thống được tổ chức thành ba khối chính (client, máy chủ ứng dụng/signaling và SFU) và hai khối hỗ trợ (PostgreSQL và coturn). Luồng điều khiển và luồng media được tách riêng để Node.js chỉ xử lý thông điệp nhỏ. Một view duy nhất định nghĩa quyền truy cập kênh; giao thức nhắn tin kết hợp mã định danh do client sinh, outbox và ràng buộc duy nhất để tránh mất và trùng tin; còn mỗi cuộc họp được gắn với một worker trong khi phần nhắn tin dùng chung giữa các tiến trình. Chương 4 trình bày cách hiện thực các khối này.

CHƯƠNG 4. TRIỂN KHAI HỆ THỐNG

Chương này mô tả cách hiện thực các khối đã thiết kế ở Chương 3. Theo hướng dẫn của khoa, chương không đưa toàn bộ mã nguồn vào báo cáo mà trình bày giải thuật bằng sơ đồ và chỉ trích những đoạn mã ngắn thể hiện ý tưởng then chốt. Các đoạn mã đã được rút gọn, và chú thích trong mã được viết thêm cho báo cáo.

4.1 Tổ chức mã nguồn

Dự án là một monorepo npm workspaces viết hoàn toàn bằng TypeScript (Bảng 4.1). Mỗi chức năng (xác thực, workspace, chat, cuộc họp, công việc, tệp, hồ sơ) có route và service riêng ở máy chủ, cùng page, component và API client riêng ở trình duyệt. Hợp đồng sự kiện Socket.IO (`ClientToServerEvents`, `ServerToClientEvents`) nằm trong gói dùng chung, nhờ đó trình biên dịch kiểm tra cả hai phía của mọi sự kiện.

Bảng 4.1. Cấu trúc mã nguồn

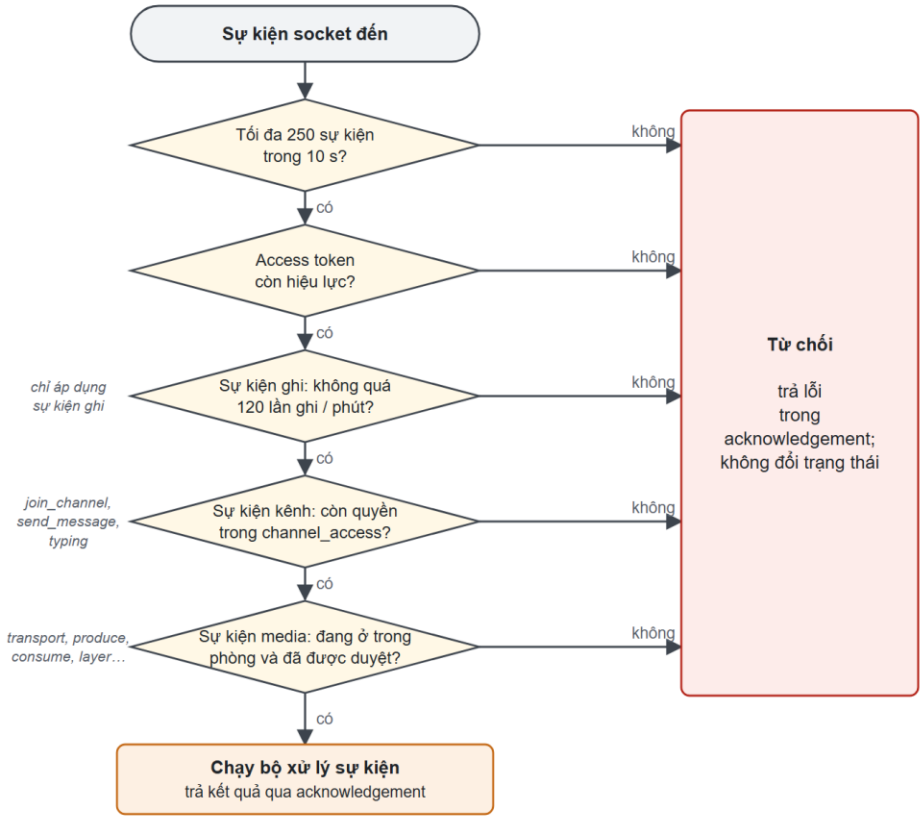
Thư mục	Nội dung
`apps/web`	Client đơn trang React: provider và layout của ứng dụng, `features/<module>`, thành phần giao diện dùng chung
`services/api`	Máy chủ Node.js: `modules/<nghiệp vụ>`, hạ tầng (cơ sở dữ liệu, thời gian thực, e-mail), 20 migration SQL
`packages/shared`	Kiểu dữ liệu và hợp đồng sự kiện Socket.IO dùng chung cho client và máy chủ
`experiments`	Phòng thí nghiệm media Mesh/SFU và công cụ phân tích gói tin (Chương 5)
`scripts`	Benchmark WebSocket, kiểm chứng sao lưu/khôi phục, dữ liệu kiểm thử
`infra`	Tệp Docker Compose cho môi trường phát triển và triển khai, cấu hình Nginx

Mã nguồn gồm khoảng 17.500 dòng TypeScript/TSX và 13.000 dòng CSS ở client, khoảng 7.000 dòng TypeScript ở máy chủ và 470 dòng ở gói dùng chung.

4.2 Lõi thời gian thực

4.2.1 Xác thực và kiểm quyền từng sự kiện

Máy chủ Socket.IO dùng chung HTTP server với REST API và có hai lớp middleware. Khi socket kết nối, access token trong handshake được xác minh và thông tin người dùng được nạp; kết nối không có token hợp lệ bị từ chối. Sau đó, trước **mọi** sự kiện, lớp middleware thứ hai thực hiện các bước kiểm tra trong Hình 4.1: giới hạn số sự kiện dồn dập trên mỗi socket, xác minh lại token, giới hạn 120 thao tác ghi mỗi phút cho mỗi người dùng, kiểm tra quyền **hiện tại** đối với kênh trong cơ sở dữ liệu (không dựa vào các phòng mà socket đã tham gia trước đó), và với signaling media, kiểm tra socket đang ở trong đúng cuộc họp và đã được cho vào. Nhờ vậy người đang ở phòng chờ hoặc đã bị mời ra không thể tạo transport, produce hay consume media.



Hình 4.1. Chuỗi kiểm quyền từng sự kiện của cổng Socket.IO

Đoạn mã 4.1 trình bày phần cốt lõi của middleware này. Tên sự kiện quyết định những bước kiểm tra nào được áp dụng.

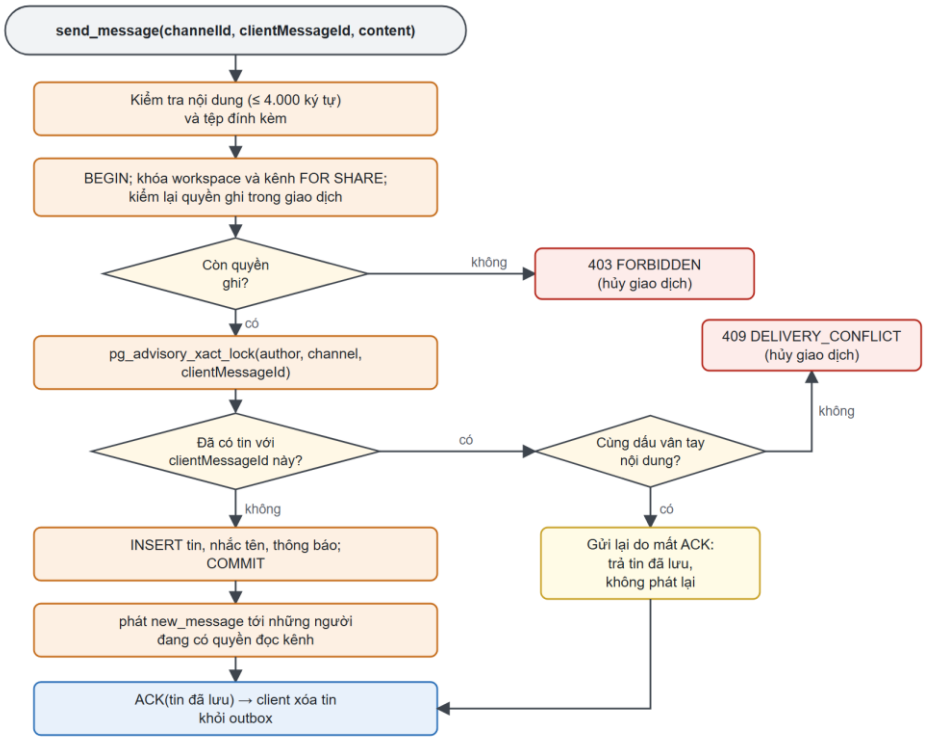
Đoạn mã 4.1. Middleware kiểm quyền từng sự kiện (services/api/src/infrastructure/realtime/socket.ts, rút gọn)

```
socket.use(async ([event, data, ack], next) => {
  try {
    if (++burstCount > 250) throw new Error("Too many operations");
    await verifyAccessToken(token);                  // token có thể đã hết hạn
    if (WRITE_EVENTS.includes(event))
      await limitActor(user.id, "realtime-write", 120, 60);
    if (["join_channel", "send_message", "typing"].includes(event))
      // quyền hiện tại đọc từ cơ sở dữ liệu, không dựa vào phòng đã join
      await requireChannelAccess(user.id, data?.channelId);
    if (MEDIA_EVENTS.includes(event)) {
      await requireChannelAccess(user.id, data?.roomId, true);
      if (event !== "join_meeting" && activeMeetingRoomId !== data.roomId)
        throw new Error("You have not joined this room");
      if (event !== "join_meeting")
        // không ở phòng chờ, không bị mời ra
        await requireAdmitted(user.id, data.roomId, socket.id);
    }
    next();
  } catch (error) { reply(ack, error); }
});
```

Việc thu hồi quyền có hiệu lực ngay: máy chủ phát `access_revoked` tới phòng `user:<id>` và ngắt mọi socket của người dùng đó.

4.2.2 Lưu tin nhắn đúng một lần

Hình 4.2 là lưu đồ xử lý `send_message`. Máy chủ kiểm tra nội dung (tối đa 4.000 ký tự) và tệp đính kèm, mở giao dịch, khóa dòng workspace và kênh ở chế độ chia sẻ, rồi kiểm tra lại quyền ghi ngay trong giao dịch, để việc thu hồi quyền xảy ra đồng thời không thể chen vào giữa bước kiểm tra và bước chèn tin. Tiếp theo máy chủ lấy một advisory lock trong phạm vi giao dịch theo bộ (tác giả, kênh, clientMessageId) và tìm tin đã có cùng mã định danh. Có ba khả năng: chưa có tin nào, khi đó tin được chèn, nhắc tên và thông báo được tạo, và `new_message` được phát sau khi commit; đã có tin cùng mã với cùng dấu vân tay nội dung, nghĩa là client đang gửi lại sau khi mất acknowledgement, khi đó tin đã lưu được trả về mà **không** phát lại; hoặc đã có tin cùng mã nhưng nội dung khác, khi đó máy chủ báo xung đột.



Hình 4.2. Lưu đồ gửi tin nhắn lũy đẳng ở máy chủ

Đoạn mã 4.2 trình bày bước chống trùng trong giao dịch. Dấu vân tay nội dung (`payloadHash`) là giá trị băm SHA-256 của nội dung, tin được trả lời, tệp đính kèm, tin gốc của luồng và danh sách người được nhắc tên, được tính trước khi giao dịch bắt đầu.

Đoạn mã 4.2. Chống trùng trong giao dịch gửi tin
(services/api/src/modules/workspaces/service.ts, rút gọn)

```
const result = await withTransaction(async (client) => {
  // Giữ quyền thành viên hiện tại cho tới khi lệnh chèn được commit
  const access = await client.query(`SELECT m.user_id FROM memberships m ...
       AND EXISTS (SELECT 1 FROM channel_access a WHERE ... AND a.can_write)
       FOR SHARE OF m`, [channelId, userId]);
  if (!access.rowCount) throw new HttpError(403, "FORBIDDEN", "...");
  if (options.clientMessageId) {
    await client.query(`SELECT pg_advisory_xact_lock(hashtextextended($1, 0))`,
      [`${userId}:${channelId}:${options.clientMessageId}`]);
    const previous = await client.query(
      `SELECT ..., client_payload_hash AS "payloadHash" FROM messages
       WHERE channel_id=$1 AND author_id=$2 AND client_message_id=$3`,
       [channelId, userId, options.clientMessageId]);
    if (previous.rowCount) {
      if (previous.rows[0].payloadHash !== payloadHash)
        throw new HttpError(409, "DELIVERY_CONFLICT", "...");
      // gửi lại do mất ACK: trả tin đã lưu, không phát lại
      deduplicated = true;
      return previous;
    }
  }
  return client.query(
    `INSERT INTO messages (..., client_message_id, client_payload_hash) ...`);
});
```

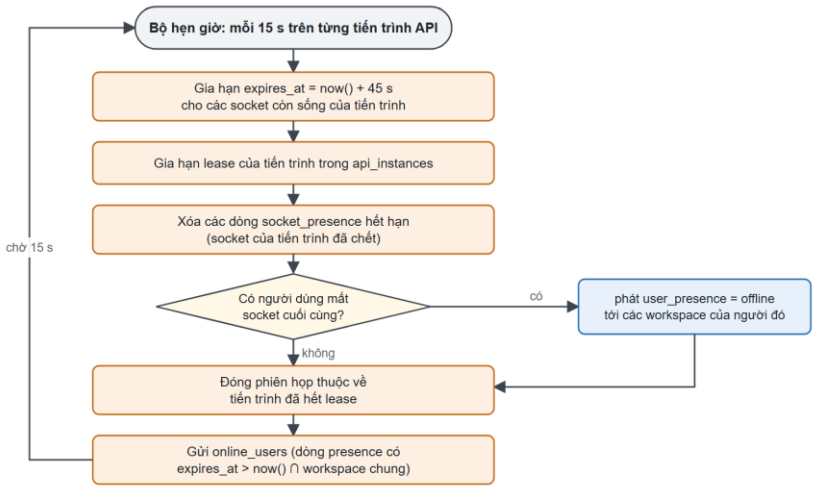
Tin mới được phát tới các phòng `user:<id>` của những người **hiện đang** có quyền đọc kênh, xác định qua view `channel_access`, chứ không phát tới phòng chung của cả workspace; nhờ vậy nội dung kênh riêng tư không bao giờ tới tay thành viên không thuộc kênh. Ở client, các tin chưa được xác nhận nằm trong outbox IndexedDB, vẫn còn sau khi tải lại trang và được gửi lại với cùng `clientMessageId`. Sau khi kết nối lại, client gọi `GET /api/channels/:id/messages?after=<cursor>` để lấy bù các tin đã lỡ rồi gộp theo mã định danh. Kết hợp với ràng buộc duy nhất trên (tác giả, clientMessageId), cơ chế này cho ngữ nghĩa *ít nhất một lần* trên đường truyền và *đúng một lần* trong cơ sở dữ liệu.

4.2.3 Trạng thái hiện diện và đang gõ

Trạng thái hiện diện được hiện thực bằng một bộ đếm kết nối trong bộ nhớ cho mỗi người dùng và bảng `socket_presence` để nhất quán giữa các tiến trình (Bảng 4.2). Hình 4.3 trình bày heartbeat chạy mỗi 15 giây trên từng tiến trình API.

Bảng 4.2. Hiện thực trạng thái hiện diện và đang gõ

Thành phần	Hiện thực
Đếm kết nối	Map từ người dùng tới số socket; trực tuyến khi số đếm tăng từ 0 lên 1, ngoại tuyến khi giảm từ 1 về 0
Trạng thái giữa các tiến trình	`socket_presence(socket_id, user_id, instance_id, expires_at)` với TTL 45 giây
Heartbeat	Mỗi 15 s: gia hạn TTL của các socket còn sống, xóa dòng hết hạn, dọn cuộc họp của tiến trình đã chết, gửi `online_users` cho từng socket
Phạm vi phát	`user_presence` chỉ tới các workspace mà người dùng là thành viên; tôn trọng tùy chọn "hiển thị hoạt động"
Đang gõ	`typing {channelId, isTyping}` → kiểm quyền → `user_typing` tới các thành viên khác của kênh; không lưu lại



Hình 4.3. Heartbeat trạng thái hiện diện chạy trên từng tiến trình API mỗi 15 giây

Vì danh sách trực tuyến được tính từ các dòng có `expires_at` ở tương lai, người dùng nối vào tiến trình A vẫn thấy được người dùng nối vào tiến trình B; khi một tiến trình bị dừng đột ngột, các dòng của nó hết hạn sau tối đa 45 giây và người dùng tương ứng chuyển sang ngoại tuyến.

4.2.4 Phân phối sự kiện giữa nhiều tiến trình

Với `SOCKET_ADAPTER=postgres`, Socket.IO dùng adapter PostgreSQL: lời gọi `io.to(room).emit(...)` được phát qua `LISTEN/NOTIFY` và các tiến trình khác phát lại cho socket cục bộ của chúng. Mỗi tiến trình giữ một dòng lease trong `api_instances`, gia hạn mỗi 15 giây và hết hạn sau 45 giây. Bài kiểm thử tích hợp `distributed.test.js` khởi chạy hai tiến trình API thật và xác nhận tin nhắn, cập nhật công việc và thông báo đều đi xuyên tiến trình.

4.3 Lõi hội nghị

4.3.1 Nhóm worker và phân bổ phòng

Room manager giữ một nhóm worker mediasoup (`MEDIASOUP_WORKERS`, mặc định hai, tối đa tám). Khi một cuộc họp bắt đầu, room manager đặt router của nó lên worker đang giữ ít phòng nhất (Đoạn mã 4.3). Nếu một worker chết, các phòng trên worker đó bị đóng, người tham gia nhận `media_failure` và được yêu cầu vào lại, các phòng trên worker khác không bị ảnh hưởng, và lượt tham gia tiếp theo sẽ tạo worker thay thế; bài kiểm thử `worker-recovery.test.js` dừng một worker thật để xác nhận hành vi này.

Đoạn mã 4.3. Chọn worker ít tải nhất (services/api/src/modules/meetings/media/room-manager.ts)

```
const worker = [...this.workers].sort(
  (a, b) =>
    (this.allocations.get(a.pid) ?? 0) - (this.allocations.get(b.pid) ?? 0),
)[0];
if (!worker) throw new Error("Media worker unavailable");
this.allocations.set(worker.pid, (this.allocations.get(worker.pid) ?? 0) + 1);
return worker;
```

Router dùng Opus 48 kHz stereo và VP8 với bitrate khởi đầu 1 Mbps. Mỗi WebRTC transport bật cả UDP và TCP, ưu tiên UDP và công bố `MEDIASOUP_ANNOUNCED_IP`; giá trị này phải là địa chỉ công khai khi triển khai thật và là địa chỉ LAN (không phải `127.0.0.1`) trong môi trường phát triển, vì lý do được giải thích ở mục 5.4.2.

4.3.2 Vòng đời producer và consumer

- `produce` kiểm tra giới hạn số nguồn của mỗi người tham gia (mặc định ba: micro, camera và màn hình), chỉ chấp nhận ba loại này và phát `new_producer` tới những người khác.
- `consume` kiểm tra `router.canConsume`, tạo consumer ở trạng thái **tạm dừng** và chỉ tiếp tục sau khi client gửi `resume_consumer`, tức là khi track đã được gắn vào giao diện, để không mất khung khóa đầu tiên.
- `consumer_layer` đặt lớp không gian ưu tiên cho consumer simulcast, còn `pause_consumer` dừng chuyển tiếp luồng không hiển thị. Khi consumer được tiếp tục, mediasoup tự yêu cầu một khung khóa mới.
- Việc đóng producer hoặc transport sẽ đóng các consumer phụ thuộc, và `peer_left` báo cho client gỡ ô video tương ứng.

4.3.3 Phiên họp và điều hành

Trạng thái phiên họp được lưu trong PostgreSQL để mọi tiến trình thấy cùng một phiên. Mỗi kênh có tối đa một phiên đang hoạt động (ràng buộc bằng partial unique index), và người vào đầu tiên trở thành host. Khi tham gia, máy chủ khóa dòng phiên, kiểm tra giới hạn số người và số tab mỗi người, rồi kiểm tra cờ khóa phòng và phòng chờ; kết quả là *được cho vào* hoặc *đang chờ*. Người đang chờ có socket nhưng chưa có peer trên SFU. Sự kiện `meeting_action` hiện thực bật phòng chờ và khóa phòng (host và co-host), cho vào, mời ra và tắt micro (host và co-host, không áp dụng lên host), chỉ định co-host, chuyển quyền host và kết thúc (chỉ host), và giơ tay (mọi người). Dòng phiên họp còn ghi lại instance media sở hữu cuộc họp, và `GET /api/channels/:id/media-location` trả về địa chỉ SFU của instance đó.

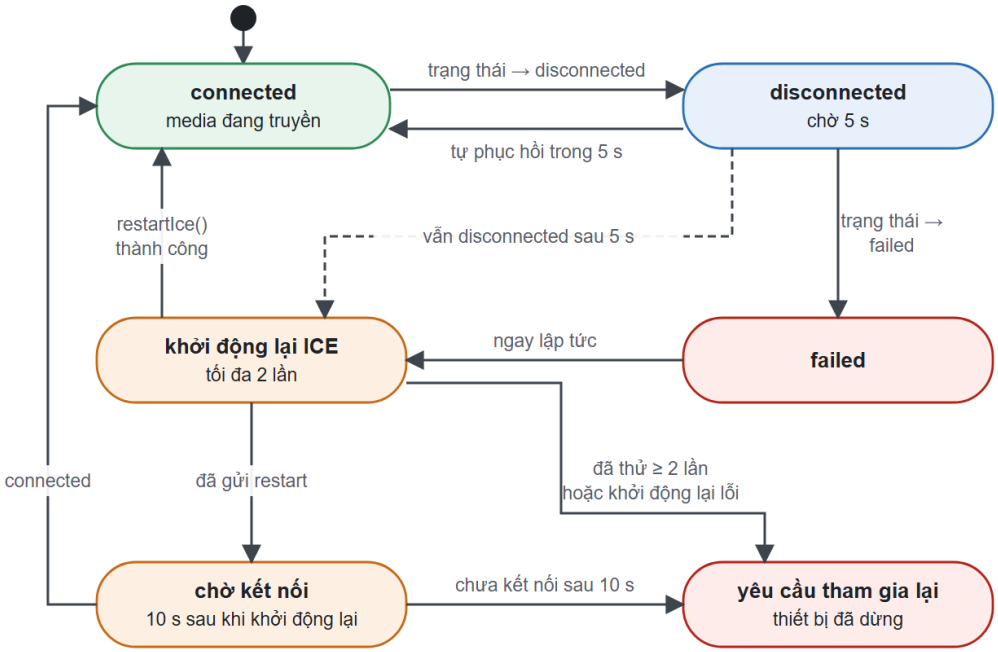
4.3.4 Vượt NAT và phục hồi ICE

Kèm mỗi transport, máy chủ trả về danh sách máy chủ ICE. Nếu có cấu hình TURN, máy chủ sinh thông tin xác thực có thời hạn như mô tả ở mục 2.2.2 (Đoạn mã 4.4).

Đoạn mã 4.4. Thông tin xác thực TURN có thời hạn (services/api/src/modules/meetings/media/ice.ts)

```
// có hiệu lực một giờ
const username = `${Math.floor(Date.now() / 1000) + 3600}:${userId}`;
return [{
  urls,                                          // lấy từ TURN_URLS
  username,
  credential: createHmac("sha1", process.env.TURN_SHARED_SECRET)
    .update(username)
    .digest("base64"),
}];
```

Ở client, mỗi transport được theo dõi trạng thái kết nối (Hình 4.4). Nếu transport ở trạng thái *disconnected* liên tục 5 giây, hoặc chuyển sang *failed*, client xin máy chủ tham số ICE mới và gọi `transport.restartIce()`, tối đa hai lần. Nếu 10 giây sau khi khởi động lại mà transport vẫn chưa kết nối, người dùng được thông báo tham gia lại.



Hình 4.4. Sơ đồ trạng thái phục hồi ICE ở client

4.4 Client hội nghị và lưới video động

4.4.1 Thu và phát media

Phòng chờ trước khi vào cho phép người dùng xem trước camera và micro, chọn thiết bị và chất lượng thu (640×360, 960×540 hoặc 1280×720). Chỉ những

nguồn đang bật mới được phát; cả bốn tổ hợp micro/camera đều đã được kiểm tra để bảo đảm nguồn bị tắt không bao giờ bị gửi đi. Camera được phát với ba bản mã hóa simulcast *q* (một phần tư độ phân giải, 150 kbps), *h* (một nửa, 400 kbps) và *f* (đầy đủ, 1.200 kbps). Chia sẻ màn hình dùng `getDisplayMedia()` và được phát thành một nguồn riêng không simulcast, tự đóng khi người dùng dừng chia sẻ từ thanh công cụ của trình duyệt. Tắt tiếng dùng `track.enabled` kết hợp `producer.pause()`, nên bật lại có hiệu lực ngay mà không phải thương lượng lại transport.

4.4.2 Thu thập số liệu QoS

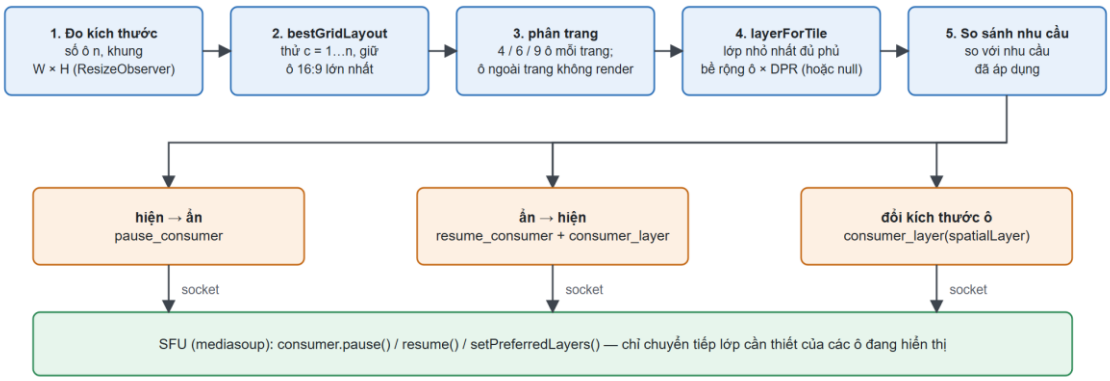
Bộ thu QoS đọc `getStats()` của cả hai transport mỗi 2,5 giây. RTT lấy từ cặp candidate đang được chọn; jitter là giá trị lớn nhất trong các luồng RTP nhận; bitrate được tính từ chênh lệch số byte trên chênh lệch thời gian; còn tỉ lệ mất gói được tính theo cửa sổ từ chênh lệch bộ đếm số gói nhận và số gói mất (Đoạn mã 4.5). Giao diện giữ 300 mẫu gần nhất (khoảng 12,5 phút) và cho phép xuất ra CSV hoặc JSON.

Đoạn mã 4.5. Tính tỉ lệ mất gói theo cửa sổ (apps/web/src/features/meetings/webrtc/qos.ts, rút gọn)

```
if (old && sample.timestamp > old.timestamp &&
    counter.received >= old.received && counter.lost >= old.lost) {
  deltaPackets += counter.received - old.received;  // bộ đếm có thể bị đặt
  deltaLost += Math.max(0, counter.lost - old.lost); // lại: bỏ qua cửa sổ đó
  hasLoss = true;
}
...
packetLoss: hasLoss && deltaPackets + deltaLost > 0
  ? (100 * deltaLost) / (deltaPackets + deltaLost)
  : null,                                    // không có dữ liệu ≠ "mất 0%"
```

4.4.3 Lưới video động và chọn lớp theo nhu cầu

Mục tiêu của lưới video là trình duyệt chỉ giải mã và vẽ những gì người dùng thực sự nhìn thấy, ở độ phân giải vừa đủ. Hình 4.5 tóm tắt giải thuật.



Hình 4.5. Lưới video theo nhu cầu: từ bố cục ô tới yêu cầu gửi SFU

1. **Bố cục tối ưu.** Với n ô trong khung có bề rộng W và chiều cao H, giải thuật thử mọi số cột c từ 1 tới n, tính số hàng r = ⌈n/c⌉ và bề rộng ô w = min((W − gap·(c−1))/c, (H − gap·(r−1))/r · 16/9), rồi giữ giá trị c cho w lớn nhất (Đoạn mã 4.6).
2. **Phân trang.** Mỗi trang chứa 4, 6 hoặc 9 ô tùy bề rộng màn hình. Ô nằm ngoài trang hiện tại không được render, nhưng âm thanh của những người đó vẫn phát qua các phần tử audio riêng.
3. **Chọn lớp cho từng ô.** Hệ thống chọn lớp simulcast nhỏ nhất mà bề rộng vẫn đủ phủ ô tính theo điểm ảnh thiết bị: với nguồn rộng 960 điểm ảnh, ô tới 264 px dùng lớp 0, tới 528 px dùng lớp 1, lớn hơn dùng lớp 2 (Đoạn mã 4.7).
4. **Đẩy nhu cầu xuống SFU.** Với mỗi người tham gia, trang họp tính nhu cầu: `null` nếu ô không hiển thị, ngược lại là số lớp. Provider của cuộc họp so sánh với nhu cầu đã áp dụng cho từng consumer và chỉ gửi phần thay đổi: `pause_consumer` khi ô biến mất, `resume_consumer` rồi `consumer_layer` khi ô xuất hiện lại, và `consumer_layer` khi ô đổi kích thước. Khi trang họp không mở (người dùng chuyển sang chat nhưng vẫn ở trong cuộc gọi), nhu cầu được gỡ và hệ thống dùng chính sách mặc định theo số camera (trên sáu → lớp 0, trên hai → lớp 1, còn lại → lớp 2).

Đoạn mã 4.6. Chọn số cột cho ô 16:9 lớn nhất (apps/web/src/features/meetings/grid-layout.ts)

```
for (let columns = 1; columns <= tiles; columns++) {
  const rows = Math.ceil(tiles / columns);
  const byWidth = (width - gap * (columns - 1)) / columns;
  const byHeight = ((height - gap * (rows - 1)) / rows) * aspect;
  const tileWidth = Math.max(0, Math.min(byWidth, byHeight));
  if (tileWidth > best.tileWidth)
    best = { columns, rows, tileWidth, tileHeight: tileWidth / aspect };
}
```

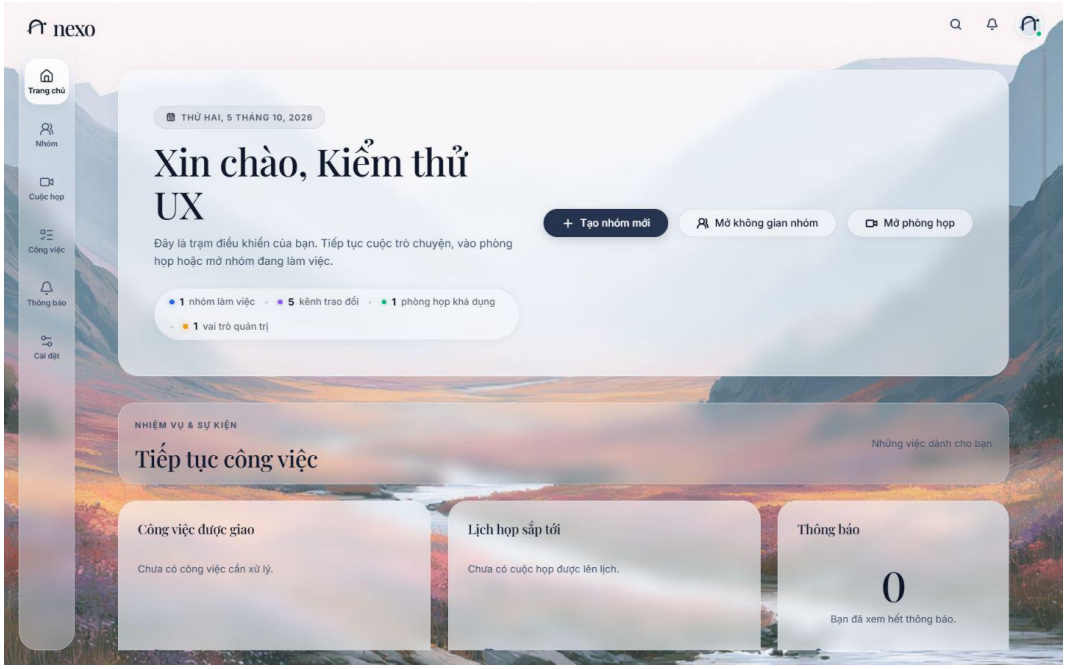
Đoạn mã 4.7. Chọn lớp simulcast cho một ô video (apps/web/src/features/meetings/grid-layout.ts)

```
export function layerForTile(
  cssWidth: number, devicePixelRatio = 1, sourceWidth = 960) {
  if (cssWidth <= 0) return null;          // không render: tạm dừng consumer
  const needed = cssWidth * devicePixelRatio;
  if (needed <= (sourceWidth / 4) * 1.1) return 0; // lớp q
  if (needed <= (sourceWidth / 2) * 1.1) return 1; // lớp h
  return 2;                                        // lớp f
}
```

Nhờ vậy, downlink của một client tỉ lệ với số lượng và kích thước các ô **đang hiển thị** chứ không còn tỉ lệ với tổng số người trong phòng. Mục 5.4.6 đo tác dụng của việc chọn lớp.

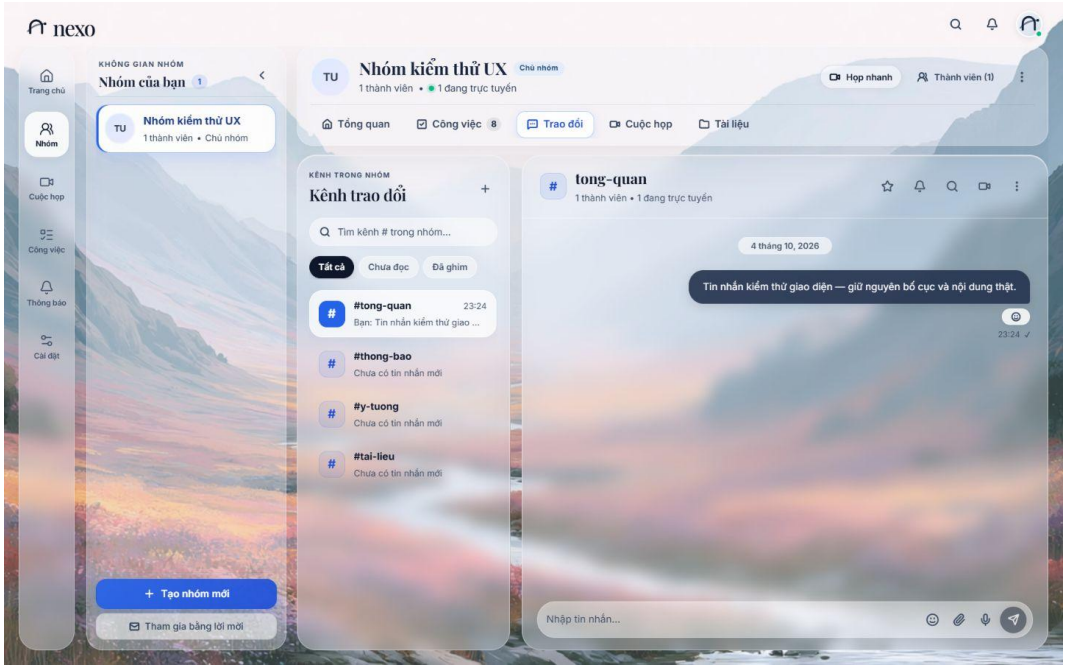
4.5 Giao diện người dùng

Client được hiện thực thành một ứng dụng đơn trang co giãn theo màn hình, giao diện tiếng Việt. Hình 4.6 là trang chủ, ưu tiên hiển thị công việc của người dùng: việc được giao, lịch họp sắp tới và thông báo, kèm lối tắt tạo nhóm mới hoặc mở phòng họp.



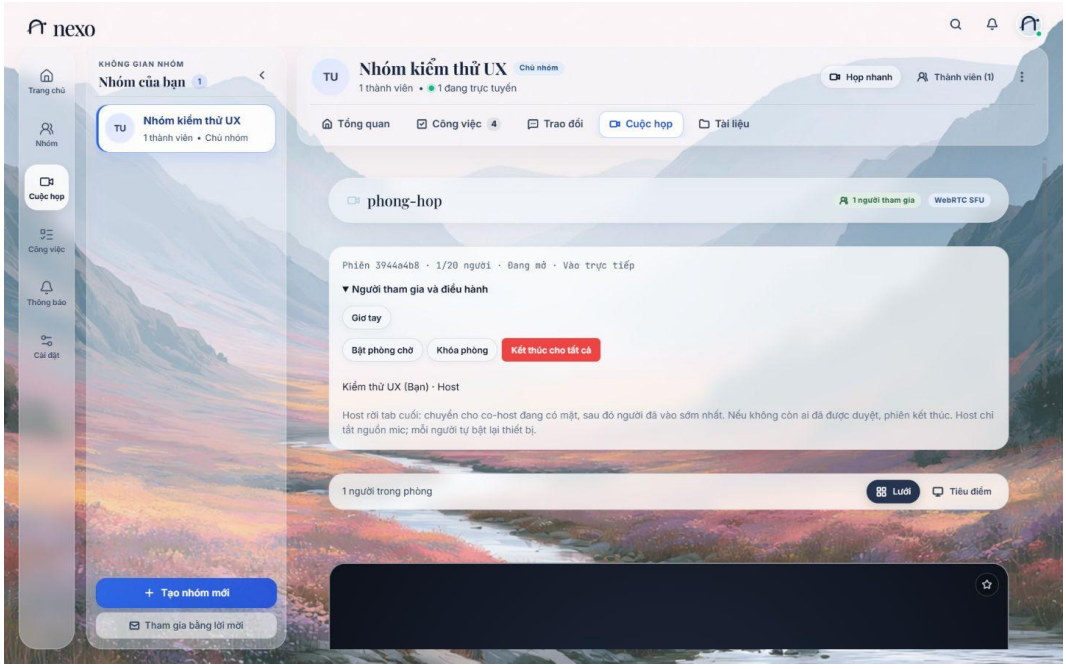
Hình 4.6. Trang chủ với công việc được giao, lịch họp sắp tới và thông báo

Hình 4.7 là một workspace với danh sách kênh và một kênh văn bản đang mở. Phần đầu trang hiển thị số thành viên và số người đang trực tuyến, các thẻ chuyển giữa tổng quan, công việc, trao đổi, cuộc họp và tài liệu của workspace, còn khung soạn tin hỗ trợ biểu tượng cảm xúc, tệp đính kèm và tin nhắn thoại.



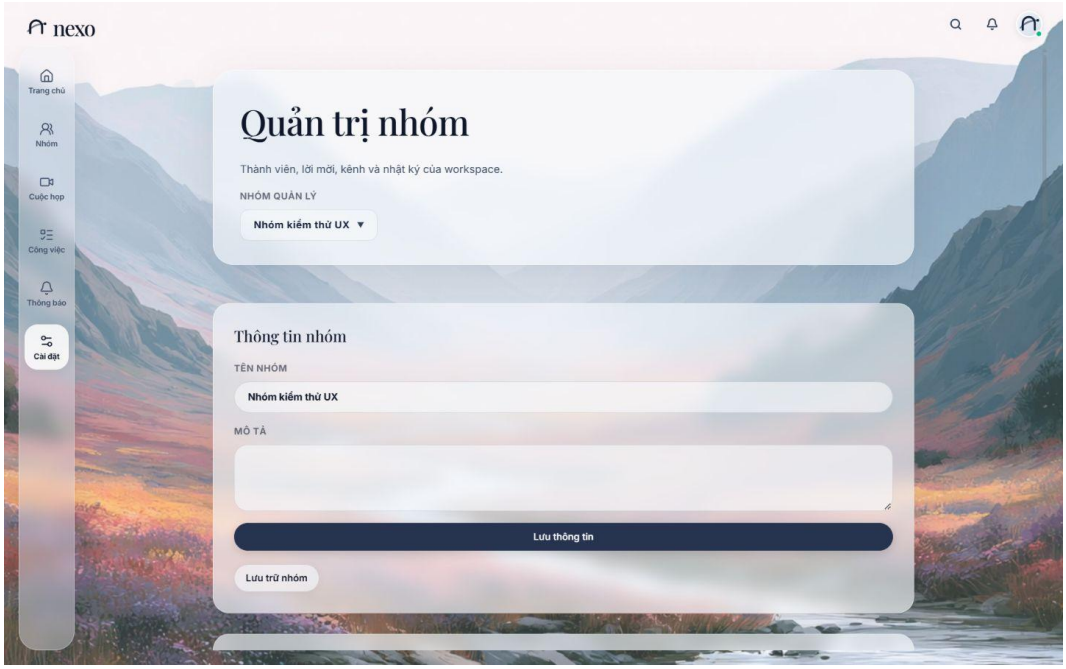
Hình 4.7. Workspace với danh sách kênh và một kênh văn bản

Hình 4.8 là một kênh video. Bảng điều hành cung cấp cho host các thao tác bật phòng chờ, khóa phòng, "kết thúc cho tất cả" và giơ tay, còn lưới video có thể chuyển giữa chế độ lưới và chế độ tiêu điểm.



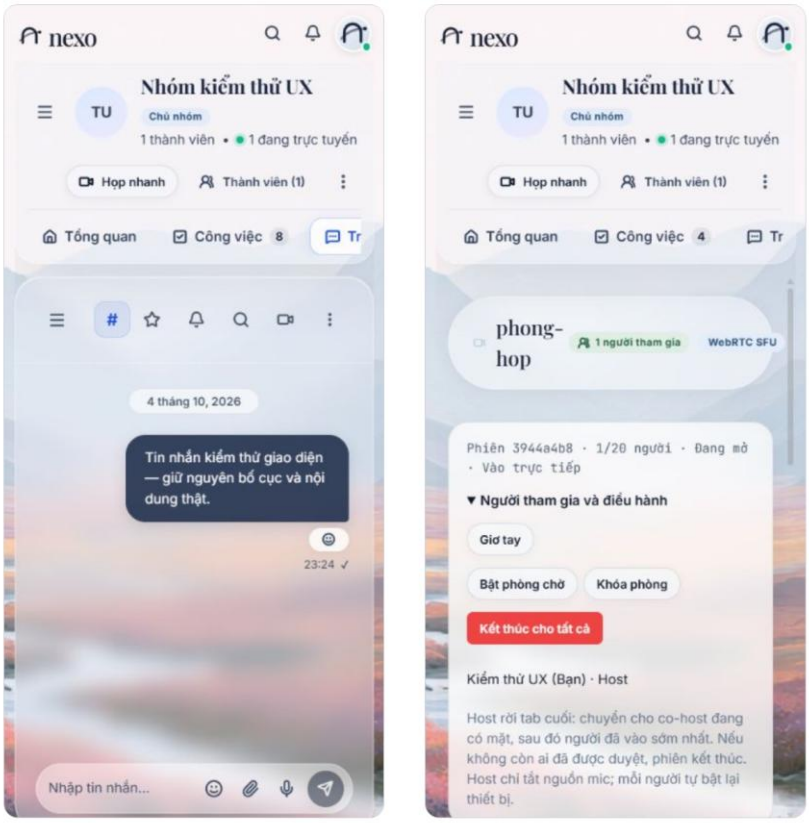
Hình 4.8. Kênh video với bảng điều hành và nút chuyển bố cục

Hình 4.9 là trang quản trị, nơi owner và admin chỉnh sửa thông tin workspace, quản lý thành viên, lời mời và kênh, lưu trữ workspace và xem nhật ký hoạt động.



Hình 4.9. Trang quản trị workspace

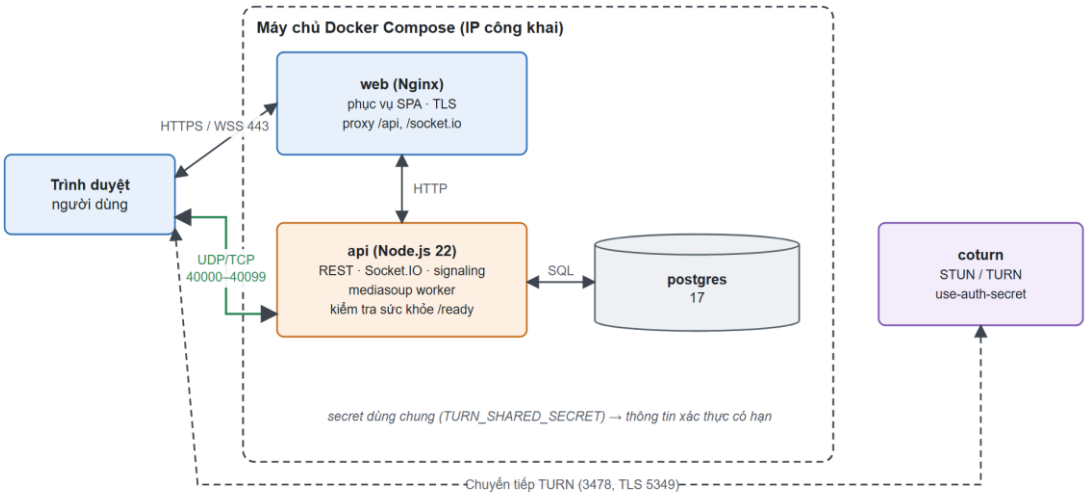
Bố cục tự điều chỉnh cho máy tính bảng và điện thoại; Hình 4.10 là giao diện chat và cuộc họp ở bề rộng 375 điểm ảnh.



Hình 4.10. Giao diện chat (trái) và cuộc họp (phải) trên màn hình điện thoại rộng 375 điểm ảnh

4.6 Triển khai

Mô hình triển khai thật, trình bày ở Hình 4.11, dùng Docker Compose với ba dịch vụ: `postgres`; `api` chạy Node.js và mediasoup, công bố trực tiếp dải cổng media 40000–40099 qua UDP và TCP; và `web` là máy chủ Nginx phục vụ client, kết thúc TLS và chuyển tiếp `/api`, `/socket.io` tới API. Một máy chủ coturn riêng cung cấp STUN/TURN với cùng secret dùng chung như API. Kiểm tra sức khỏe `/ready` chỉ đạt khi PostgreSQL truy cập được và mọi migration đã được áp dụng. Các biến môi trường cho chế độ nhiều tiến trình (`SOCKET_ADAPTER=postgres`, `API_INSTANCE_ID`, `SFU_PUBLIC_URL`) đã có sẵn, nhưng mô hình nhiều node chưa được nghiệm thu trên hạ tầng thật (mục 5.7).



Hình 4.11. Mô hình triển khai bằng Docker Compose

4.7 Tóm tắt chương

Máy chủ kiểm quyền mọi sự kiện Socket.IO theo trạng thái hiện tại của cơ sở dữ liệu, lưu mỗi tin nhắn đúng một lần nhờ advisory lock và ràng buộc duy nhất, và giữ trạng thái hiện diện nhất quán giữa các tiến trình bằng một bảng có TTL. Lõi hội nghị đặt mỗi cuộc họp lên worker mediasoup ít tải nhất, khởi tạo consumer ở trạng thái tạm dừng, tự phục hồi ICE và cấp thông tin xác thực TURN có thời hạn. Client phát video simulcast và chỉ yêu cầu SFU gửi những lớp mà các ô đang hiển thị cần đến. Chương 5 đánh giá các cơ chế này.

CHƯƠNG 5. THỰC NGHIỆM VÀ ĐÁNH GIÁ

Chương này đánh giá hệ thống theo các yêu cầu ở mục 3.1.3. Trước hết chương mô tả môi trường thử nghiệm, các đại lượng đo và hệ thống chuẩn để so sánh (mục 5.1), sau đó trình bày kết quả kiểm thử chức năng và độ tin cậy (mục 5.2), độ trễ của lõi nhắn tin khi chịu tải (mục 5.3) và thực nghiệm chính so sánh SFU của hệ thống với hệ thống chuẩn Mesh (mục 5.4). Mục 5.5 so sánh hệ thống với các nền tảng hiện có, mục 5.6 đối chiếu với yêu cầu và mục 5.7 thảo luận các giới hạn của phép đo. Mọi con số đều lấy từ dữ liệu gốc lưu trong kho mã tại `docs/evidence/`; những giá trị được suy ra từ dữ liệu gốc (ví dụ tổng hay tỉ lệ) được ghi chú là *tính lại*.

5.1 Thiết lập thực nghiệm

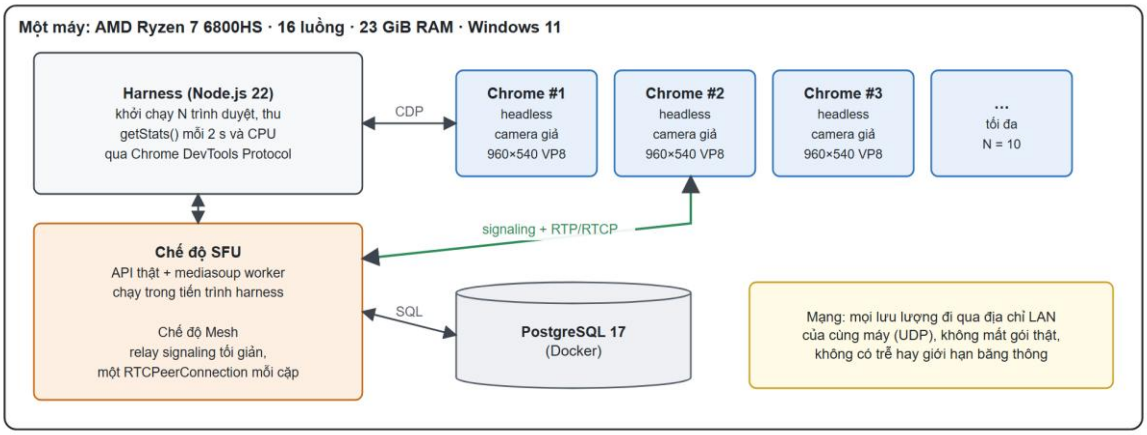
5.1.1 Môi trường và quy mô thử nghiệm

Mọi thực nghiệm đều chạy trên một máy phát triển (Bảng 5.1). Hình 5.1 mô tả cách tổ chức thực nghiệm media: một chương trình điều phối (harness) viết bằng Node.js khởi chạy mỗi người tham gia thành **một tiến trình Chrome headless riêng** với camera và micro giả, cho vào phòng họp, rồi lấy mẫu thống kê WebRTC và mức dùng CPU qua Chrome DevTools Protocol. Tối đa mười trình duyệt chạy đồng thời trong thực nghiệm media, và tối đa 200 client Socket.IO trong thực nghiệm nhắn tin.

Bảng 5.1. Môi trường thử nghiệm

Thành phần	Giá trị
Phần cứng	AMD Ryzen 7 6800HS, 8 nhân / 16 luồng, 23,3 GiB RAM
Hệ điều hành	Windows 11 (x64)
Phần mềm	Node.js 22.19, PostgreSQL 17.11 (Docker), Google Chrome (headless), mediasoup 3
Nguồn media	Camera giả của Chromium, yêu cầu 960×540 ở 30 fps (thực tế phát 20 fps), VP8 và Opus
Mạng	Mọi trình duyệt, API và SFU trên cùng một máy; media gửi tới địa chỉ LAN của máy qua UDP

Thành phần	Giá trị
Quy mô	Media: 2–10 trình duyệt mỗi phòng. Nhắn tin: 50, 100 và 200 socket trên hai tiến trình API



Hình 5.1. Môi trường thực nghiệm media

Việc chạy mọi thứ trên một máy có một hệ quả được dùng xuyên suốt chương này: đường truyền không có độ trễ, mất gói hay giới hạn băng thông thật, nên RTT, jitter và tỉ lệ mất gói phản ánh **tải xử lý** chứ không phản ánh chất lượng mạng; và khi tổng nhu cầu CPU tiến gần năng lực của máy, mọi chỉ số đều xấu đi do tranh chấp tài nguyên. Mục 5.7 thảo luận giới hạn này.

5.1.2 Các đại lượng đo

Các đại lượng media được đọc từ `getStats()` mỗi 2 giây, theo định nghĩa ở mục 2.2.4 và mục 4.4.2:

- **Uplink và downlink** (kbps): chênh lệch số byte (tải và header RTP) trên chênh lệch thời gian.
- **CPU client** (%): tổng thời gian CPU của mọi tiến trình thuộc một trình duyệt; 100% tương ứng một lõi.
- **RTT** (ms) của cặp candidate ICE đang được chọn; **jitter âm thanh và video** (ms); **tỉ lệ mất gói** (%) theo cửa sổ.

- **Tốc độ khung hình nhận**, **độ trễ jitter buffer**, **số lần đóng băng hình** và số **bộ mã hóa đang hoạt động** (số luồng video gửi đi có bitrate trên 1 kbps).
- **CPU máy chủ**: CPU của worker mediasoup và của tiến trình Node.js.

Mỗi mức N được đo trong 60 giây sau 30 giây khởi động để bộ ước lượng băng thông ổn định, tức 30 mẫu cho mỗi client. Mỗi mức media được lặp lại **ba lần**; các bảng ghi giá trị trung bình qua các lượt và, khi có nhiều lượt, độ lệch chuẩn mẫu giữa các lượt (± SD). Độ trễ tin nhắn được báo cáo bằng các phân vị p50, p95 và p99 (nearest-rank) trên toàn bộ mẫu.

5.1.3 Hệ thống chuẩn để so sánh

Hướng dẫn của khoa yêu cầu lấy một hệ thống làm chuẩn để so sánh với hệ thống của mình. Với lõi media, hệ thống chuẩn là một hiện thực **Mesh ngang hàng toàn phần**, kiến trúc mà một ứng dụng WebRTC nhỏ sử dụng khi không có máy chủ media (mục 2.3.1). Hiện thực hệ thống chuẩn ngay trong cùng phòng thí nghiệm giúp phép so sánh công bằng: hai kiến trúc dùng cùng trình duyệt, cùng nguồn giả, cùng codec, cùng máy và cùng mã đo, chỉ khác nhau ở cấu trúc kết nối. Client Mesh mở một `RTCPeerConnection` cho mỗi cặp người tham gia qua một relay signaling tối giản và gửi một bản mã hóa với tối đa 1.200 kbps cho mỗi kết nối; client SFU chính là `MediasoupClientService` của nexo với ba lớp simulcast và chính sách chọn lớp mặc định. Mục 5.5 bổ sung phần so sánh định tính giữa nexo với các sản phẩm hiện có.

5.2 Kiểm thử chức năng và độ tin cậy

Bảng 5.2 tóm tắt các bộ kiểm thử tự động. Các bộ kiểm thử tích hợp chạy trên cơ sở dữ liệu PostgreSQL thật và worker mediasoup thật.

Bảng 5.2. Các bộ kiểm thử tự động

Bộ kiểm thử	Phạm vi	Kết quả
Kiểm thử đơn vị frontend (`npm test`, web)	URL định danh, bố cục lưới, tính toán QoS, thao tác media, giơ tay,	Đạt 39 / 39

Bộ kiểm thử	Phạm vi	Kết quả
	thông báo khi rớt cuộc gọi, trạng thái tương tác	
Kiểm thử đơn vị API (`npm test`, api)	Xác thực, kiểm tra sức khỏe, ảnh đại diện Google, hợp đồng; các bộ DB/SFU được bỏ qua có chủ đích ở lệnh mặc định	Đạt 17, bỏ qua 8 có chủ đích
Kiểm thử tích hợp với PostgreSQL và mediasoup (`npm run test:sfu`)	Quản trị và phân quyền, hội thoại, công việc, phiên họp, lưu trữ tệp, hai tiến trình API, phục hồi worker	Đạt 9 / 9, không bỏ qua
Kiểm tra tĩnh	`typecheck`, `lint`, `build` toàn bộ workspace	Không có lỗi

Các bài kiểm thử xác nhận những tính chất tin cậy yêu cầu ở mục 3.1.3:

- **Lũy đẳng khi gửi tin.** Sáu yêu cầu đồng thời với cùng `clientMessageId` chỉ tạo đúng một dòng; bỏ acknowledgement rồi gửi lại chỉ tạo đúng một lần phát `new_message`; phân trang bằng con trỏ `after` không bao giờ trả một tin hai lần.
- **Thu hồi quyền.** Sau khi quyền vào kênh riêng tư bị thu hồi, cả việc gửi lại lẫn việc lấy bù tin sau khi kết nối lại đều bị từ chối.
- **Quản trị.** Quyền được kiểm tra chéo trên REST, socket, tìm kiếm, tải tệp, lịch họp và media; lời mời sai e-mail, hết hạn, bị thu hồi hoặc được chấp nhận đồng thời đều được xử lý đúng; chuyển quyền sở hữu hoạt động và cơ sở dữ liệu từ chối xóa owner duy nhất.
- **Tìm kiếm.** Kiểm tra với tiếng Việt, các ký tự `%` và `_`, và việc mở một tin nằm ngoài 100 tin gần nhất; kế hoạch truy vấn trên 10.000 tin xác nhận index theo dòng thời gian hiện có là đủ ở quy mô này.
- **Cuộc họp.** Phòng chờ, khóa phòng, mời ra, kết thúc và giới hạn số người, số tab đều được kiểm thử; cả bốn tổ hợp micro/camera được kiểm tra để bảo đảm nguồn bị tắt không bao giờ được phát.

- **Chịu lỗi.** Với hai worker, dừng một worker không ảnh hưởng tới người tham gia trên worker còn lại, và lượt tham gia tiếp theo tạo worker thay thế.
- **Hai tiến trình API.** Với adapter PostgreSQL, tin nhắn, công việc và thông báo đi xuyên tiến trình, và việc thu hồi quyền có hiệu lực trên cả hai.
- **Migration và khôi phục.** 20 migration có checksum; hai trình chạy migration khởi động cùng lúc trên một cơ sở dữ liệu không xung đột; bản sao lưu được khôi phục sang cơ sở dữ liệu mới giữ nguyên schema và dữ liệu Unicode.

Các bài kiểm thử này xác nhận tính đúng của luồng điều khiển và phân quyền; chúng không đo chất lượng RTP, vốn là nội dung của mục 5.4.

5.3 Độ trễ của lõi nhắn tin khi chịu tải

5.3.1 Kịch bản

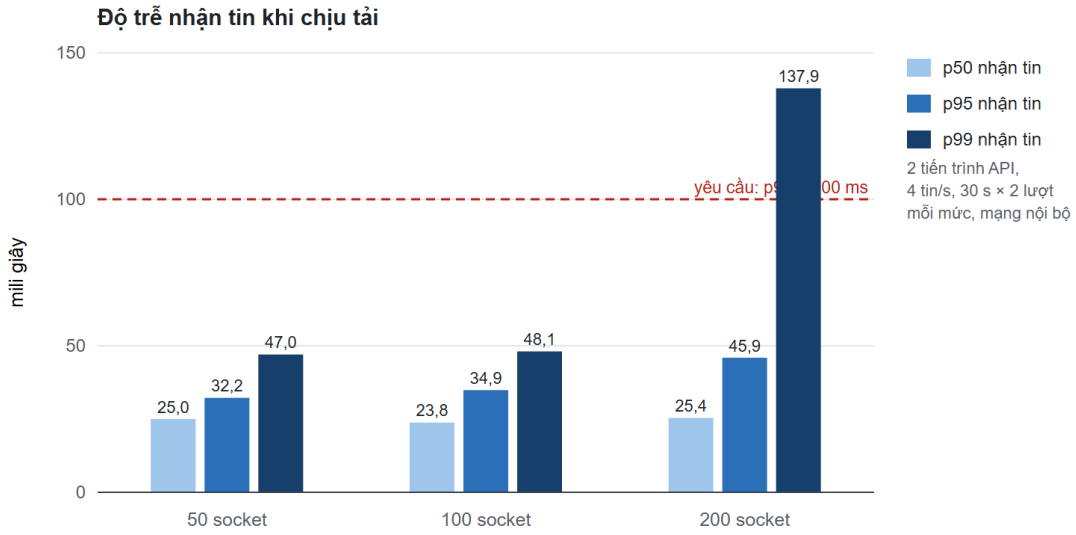
Hai tiến trình API dùng chung PostgreSQL qua adapter PostgreSQL (pool tám kết nối mỗi tiến trình); bốn workspace, mỗi workspace hai kênh văn bản được tạo sẵn. Mỗi mức (50, 100 và 200 socket) chạy 30 giây và lặp lại hai lần, với **tổng 4 tin mỗi giây** do bốn người gửi đồng thời. Trong mỗi lượt, acknowledgement bị cố ý bỏ sáu lần và tin được gửi lại với cùng mã định danh, và hai tin được gửi trong lúc một client đang ngoại tuyến để kiểm tra việc lấy bù tin. Phần lớn socket là người nhận, nên kịch bản mô phỏng một kênh đông người đọc chứ không phải 200 người cùng gõ liên tục.

5.3.2 Kết quả

Bảng 5.3 và Hình 5.2 trình bày độ trễ đo được. *ACK* là thời gian từ lúc phát tin tới khi nhận acknowledgement thành công; *nhận tin* là thời gian từ lúc phát tin tới khi từng người nhận nhận được `new_message`, có trọng số theo số người nhận.

Bảng 5.3. Độ trễ tin nhắn tính bằng mili giây (hai lượt mỗi mức)

Số socket	ACK p50 / p95 / p99	Nhận tin p50 / p95 / p99	Tin duy nhất / lượt nhận	Lỗi / trùng
50	24,99 / 39,88 / 53,79	25,01 / 32,23 / 47,02	244 / 3.052	0 / 0
100	23,69 / 46,09 / 56,81	23,78 / 34,87 / 48,07	244 / 6.096	0 / 0
200	25,23 / 48,58 / 140,26	25,38 / 45,93 / 137,86	244 / 12.196	0 / 0



Hình 5.2. Các phân vị độ trễ nhận tin ở mức 50, 100 và 200 socket

Tổng cộng 732 tin tạo ra 21.344 lượt nhận, và số dòng trong cơ sở dữ liệu luôn bằng số tin duy nhất ở mọi lượt: **không có tin nào bị mất hay bị trùng**, dù 36 lần gửi phải lặp lại sau khi mất acknowledgement và sáu client đã kết nối lại. Mỗi lần kết nối lại lấy bù đủ hai tin bị lỡ qua REST trong 21,5–54,9 ms. Độ trễ trung vị gần như không đổi (23,7–25,4 ms) khi số socket tăng gấp bốn; phân vị 95 tăng từ 32 lên 46 ms, và chỉ phân vị 99 ở mức 200 socket tăng lên khoảng 140 ms.

Bảng 5.4 trình bày tài nguyên sử dụng. Mỗi tiến trình API dùng dưới 6% một lõi và dưới 120 MiB bộ nhớ. Số yêu cầu phải chờ kết nối cơ sở dữ liệu tăng theo số socket (13, 31 và 120) nhưng chỉ xuất hiện trong đợt kết nối dồn dập ban đầu và biến mất ở các mẫu sau. Vì vậy đợt kết nối ban đầu cùng các cập nhật trạng thái hiện diện đi kèm là điểm cần tối ưu đầu tiên trước khi tăng tải, thay vì chỉ đơn giản tăng kích thước pool.

Bảng 5.4. Tài nguyên trong bài kiểm tra tải nhắn tin (giá trị lớn nhất quan sát được)

Số socket	RSS mỗi API (MiB)	CPU mỗi API (% một lõi)	Số yêu cầu chờ pool lớn nhất	CPU / bộ nhớ PostgreSQL
50	98,8	5,60	13	5,1–5,2% / 107–112 MiB
100	95,7	3,75	31	5,0–6,0% / 111–113 MiB
200	115,2	4,66	120	4,8–5,6% / 112–113 MiB

5.4 So sánh Mesh và SFU

Đây là thực nghiệm chính của đồ án, nhằm kiểm chứng lựa chọn kiến trúc ở mục 2.3.4: khi số người tham gia N tăng, kiến trúc nào giữ được chất lượng cuộc gọi với chi phí chấp nhận được?

5.4.1 Thiết kế thực nghiệm

Bảng 5.5 tóm tắt cấu hình của hai kiến trúc. Cả hai được đo theo cùng một quy trình: harness tạo người dùng và workspace thử nghiệm mới cho từng mức, khởi chạy N trình duyệt, chờ 30 giây, đo trong 60 giây rồi xóa dữ liệu thử nghiệm. Mỗi kiến trúc được đo ba lần ở mỗi mức; các lượt Mesh dừng ở N = 8 vì máy đã bão hòa từ N = 6, còn các lượt SFU đi tới N = 10. Hai cấu hình SFU bổ sung giúp tách riêng tác dụng của simulcast: chỉ một bản mã hóa thay vì ba lớp, và ép mọi consumer về lớp thấp nhất.

Bảng 5.5. Cấu hình của hai kiến trúc được so sánh

Mục	Hệ thống chuẩn Mesh	SFU (nexo)
Cấu trúc kết nối	Một RTCPeerConnection cho mỗi cặp, relay signaling tối giản	API thật và worker mediasoup thật; mã client của sản phẩm
Mã hóa video	Một bản mã hóa, tối đa 1.200 kbps mỗi kết nối	Simulcast q/h/f = 150/400/1.200 kbps

Mục	Hệ thống chuẩn Mesh	SFU (nexo)
Lớp nhận	–	Chính sách mặc định: trên 6 camera → lớp 0, trên 2 → lớp 1, còn lại → lớp 2
Nguồn	Camera giả 960×540, 20 fps, VP8; âm thanh Opus	Như Mesh
Hiển thị video nhận	Tắt (giải mã nhưng không vẽ)	Tắt
Các mức N	2, 4, 6, 8	2, 4, 6, 8, 10
Số lần lặp	3 lượt × 60 s mỗi mức (khởi động 30 s)	3 lượt × 60 s mỗi mức (khởi động 30 s)

Video nhận được giải mã nhưng không được vẽ. Ở chế độ headless, trình duyệt vẽ video bằng phần mềm, tốn khoảng một lõi cho mỗi client và sẽ che lấp chi phí truyền tải và codec đang cần so sánh; chi phí vẽ là như nhau ở cả hai kiến trúc với cùng số ô, và trên thiết bị thật việc vẽ do GPU đảm nhận.

5.4.2 Các sai lệch đo lường

Hai hiện tượng sẽ làm sai lệch phép so sánh nếu không được phát hiện và loại bỏ.

SFU công bố địa chỉ loopback. Khi SFU công bố `127.0.0.1`, trình duyệt không thu thập loopback candidate, việc kiểm tra kết nối UDP thất bại và ICE rơi xuống candidate TCP thụ động của mediasoup. RTP chạy trên TCP bị chặn đầu hàng đợi (head-of-line blocking), sinh ra NACK dày đặc và jitter cao dù không hề có mất gói thật. Vì vậy harness công bố địa chỉ LAN của máy, và mọi kết quả SFU dưới đây đều đi qua UDP (cặp candidate được chọn được ghi kèm mỗi mẫu). Cấu hình phát triển của ứng dụng cũng phải dùng địa chỉ LAN vì cùng lý do.

Gói thăm dò băng thông. mediasoup gửi gói đệm trên một luồng riêng (SSRC 1234) để dò băng thông còn trống trước khi nâng lớp cho consumer. Luồng này có "mất gói" và jitter khoảng 200 ms giả tạo, trong khi các luồng media thật không mất

gói và jitter gần bằng không; thống kê consumer phía SFU cũng ghi nhận không mất gói và không có NACK. Số byte của luồng này vẫn được tính vào downlink vì chúng thật sự đi qua đường truyền, nhưng luồng này bị loại khỏi các chỉ số chất lượng. Nếu không hiệu chỉnh, SFU sẽ bị đánh giá oan là kém hơn Mesh.

5.4.3 Kết quả: băng thông

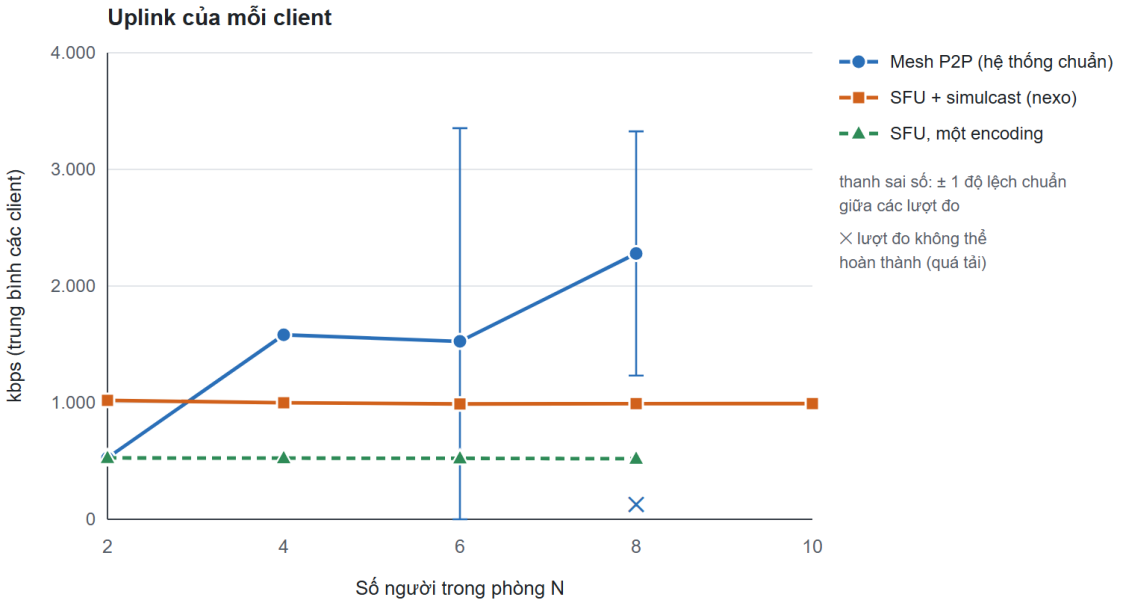
Bảng 5.6 và Bảng 5.7 trình bày kết quả của hai kiến trúc, còn Hình 5.3 đến Hình 5.7 vẽ các chỉ số chính theo N với thanh sai số bằng một độ lệch chuẩn giữa các lượt.

Bảng 5.6. Kết quả của hệ thống chuẩn Mesh (trung bình ± độ lệch chuẩn giữa các lượt)

N	Uplink (kbps)	Downlink (kbps)	CPU (%)	Encoder	RTT (ms)	Jitter video (ms)	Mất gói (%)	FPS
2	526 ± 1	526 ± 1	26,0 ± 2,0	1,0 ± 0,0	0,87 ± 0,10	0,00 ± 0,00	0,00 ± 0,00	19,9 ± 0,1
4	1.582 ± 1	1.582 ± 1	81,1 ± 4,1	3,0 ± 0,0	0,88 ± 0,03	1,09 ± 0,66	0,00 ± 0,00	20,0 ± 0,1
6	1.525 ± 1.828	658 ± 599	202,1 ± 3,1	1,8 ± 0,0	3.457 ± 850,4	383,7 ± 56,99	0,23 ± 0,25	10,1 ± 3,8
8	2.279 ± 1.047	1.845 ± 807	154,0 ± 6,1	1,9 ± 0,3	5.432 ± 4.275	633,4 ± 291,5	0,01 ± 0,01	14,7 ± 0,1

Bảng 5.7. Kết quả của SFU có simulcast (trung bình ± độ lệch chuẩn giữa các lượt)

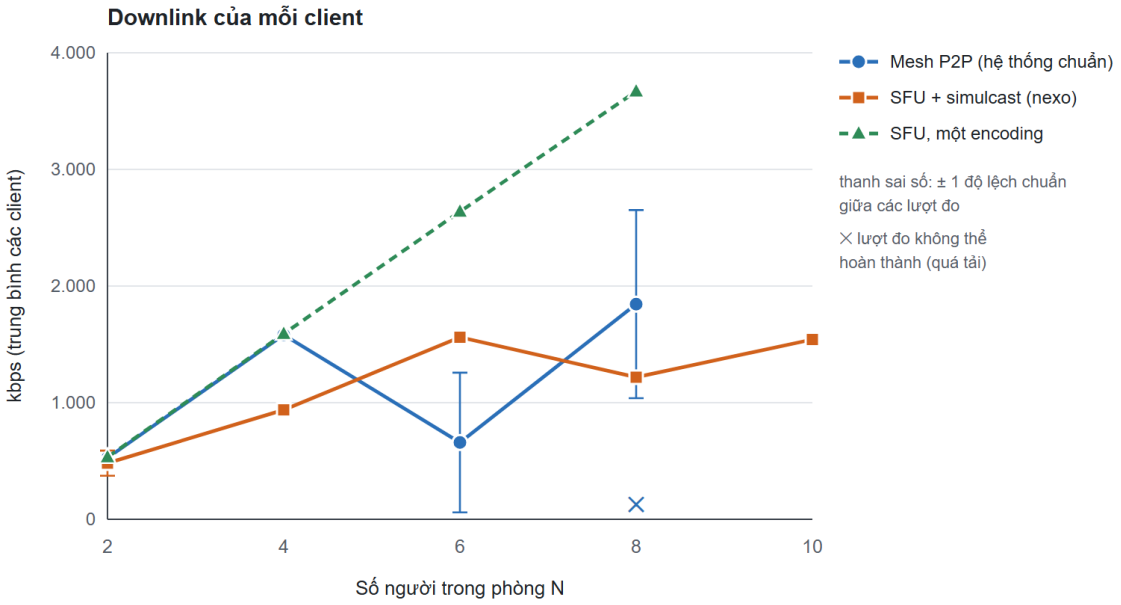
N	Uplink (kbps)	Downlink (kbps)	CPU (%)	Encoder	RTT (ms)	Jitter video (ms)	Mất gói (%)	FPS
2	1.020 ± 5	481 ± 108	25,8 ± 3,7	3,0 ± 0,0	0,19 ± 0,10	0,33 ± 0,08	0,00 ± 0,00	17,2 ± 2,3
4	999 ± 13	938 ± 12	33,8 ± 1,2	3,0 ± 0,0	0,29 ± 0,10	0,31 ± 0,20	0,00 ± 0,00	20,0 ± 0,0
6	989 ± 3	1.561 ± 9	41,7 ± 0,9	3,0 ± 0,0	0,47 ± 0,11	0,75 ± 0,19	0,00 ± 0,00	20,0 ± 0,0
8	992 ± 6	1.218 ± 19	40,9 ± 0,2	3,0 ± 0,0	0,53 ± 0,03	0,82 ± 0,07	0,00 ± 0,00	20,0 ± 0,0
10	992 ± 2	1.542 ± 2	47,1 ± 1,8	3,0 ± 0,0	1,00 ± 0,11	1,83 ± 0,41	0,00 ± 0,00	20,0 ± 0,0



Hình 5.3. Uplink của mỗi client theo số người tham gia

Hình 5.3 là kết quả chính. Uplink của Mesh tăng theo số kết nối: mỗi kết nối mang khoảng 526 kbps, nên một client tải lên 526 ± 1 kbps ở N = 2 và 1.582 ± 1 kbps ở N = 4 với ba bộ mã hóa, đúng bằng (N−1)·b trong Bảng 2.2. Uplink của SFU gần như phẳng: 1.020 ± 5 kbps ở N = 2 và 992 ± 2 kbps ở N = 10, luôn với ba bộ mã hóa (ba lớp simulcast). Ở N = 2, client SFU tải lên nhiều hơn client Mesh vì phải gửi ba lớp thay vì một; đây là chi phí cố định của simulcast, cái giá để uplink không phụ thuộc N. Từ số liệu Mesh, hai kết nối ở N = 3 sẽ cần khoảng 1.053 kbps (*tính lại* bằng ngoại suy tuyến tính, không đo), nên điểm hòa vốn nằm quanh N = 3; từ N = 4, SFU tải lên ít hơn Mesh 37% (*tính lại*).

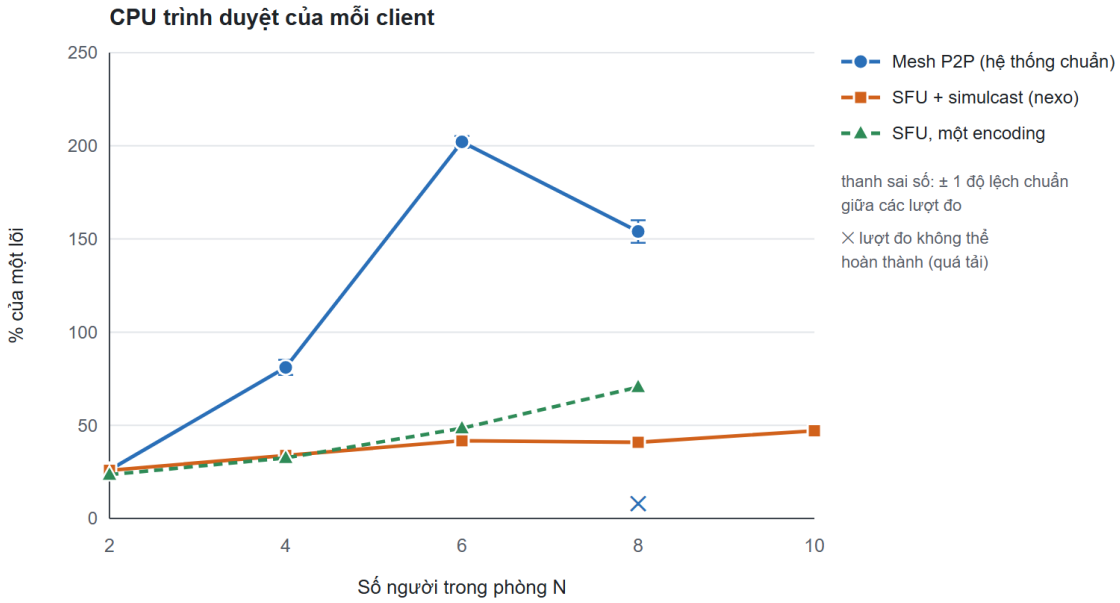
Giá trị Mesh ở N = 6 và 8 không phản ánh nhu cầu lý thuyết (năm và bảy luồng, khoảng 2.636 và 3.691 kbps, *tính lại*): số bộ mã hóa thật sự đang phát giảm xuống trung bình 1,8 và 1,9 vì các kết nối bị nghẽn, và chênh lệch giữa các lượt rất lớn. Đây là trạng thái quá tải chứ không phải trạng thái ổn định.



Hình 5.4. Downlink của mỗi client theo số người tham gia

Hình 5.4 trình bày downlink. Với Mesh, mỗi luồng nhận tốn khoảng 526 kbps và tổng downlink tăng theo N−1. Với SFU, downlink đi theo chính sách chọn lớp chứ không theo N: một luồng ở lớp 2 tốn 481 kbps ở N = 2; ở N = 4 và 6, các consumer chuyển sang lớp 1 và mỗi luồng tốn khoảng 312 kbps (*tính lại*); khi có hơn sáu camera, các consumer chuyển sang lớp 0, nên downlink ở N = 8 (1.218 kbps cho bảy luồng) thấp hơn ở N = 6, và ở N = 10 chín luồng tốn 1.542 kbps. Như vậy downlink của client SFU luôn bị chặn trên dù phòng đông lên, đúng với mục đích của simulcast và chính sách chọn lớp.

5.4.4 Kết quả: CPU



Hình 5.5. CPU trình duyệt của mỗi client theo số người tham gia

Hình 5.5 so sánh CPU của một trình duyệt. Với Mesh, CPU tăng từ 26,0 ± 2,0% lên 81,1 ± 4,1% một lõi khi N tăng từ 2 lên 4, tức khoảng 3,1 lần (*tính lại*), phù hợp với việc số bộ mã hóa và giải mã của mỗi client tăng từ một lên ba. Ở N = 6, một trình duyệt Mesh dùng trung bình 202,1%, tức hơn hai lõi, vượt xa mức một thiết bị người dùng thông thường có thể dành cho một cuộc gọi. Với SFU, CPU tăng chậm từ 25,8 ± 3,7% ở N = 2 lên 47,1 ± 1,8% ở N = 10, vì số bộ mã hóa cố định và chỉ số bộ giải mã tăng; ở N = 4, client SFU dùng ít hơn client Mesh 58% CPU (*tính lại*).

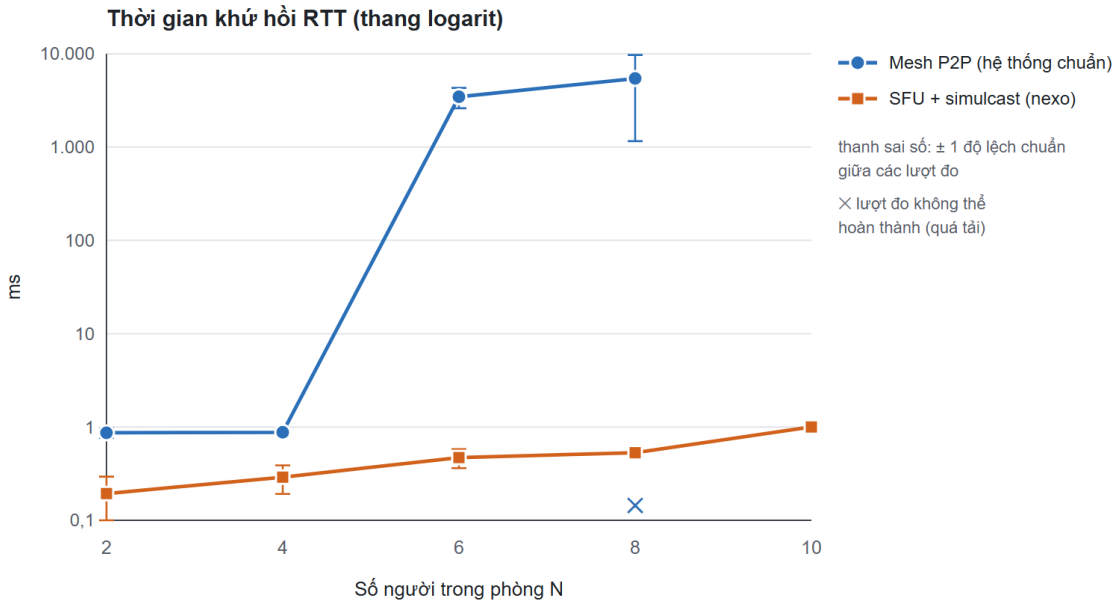
Bảng 5.8 cho thấy phần chi phí được chuyển sang máy chủ. Worker mediasoup dùng 2,7% một lõi ở N = 2 và 38,9% ở N = 10; mức này tăng nhanh hơn tuyến tính vì số luồng cần chuyển tiếp tăng theo N(N−1). Tiến trình Node.js, nơi chạy signaling và API, chỉ dùng từ 0,1% đến 1,0% một lõi với 86–109 MiB bộ nhớ, xác nhận việc tách luồng điều khiển và luồng media ở mục 3.2.2: Node.js không bao giờ chạm vào gói media.

Bảng 5.8. Chi phí phía máy chủ của SFU (trung bình giữa các lượt)

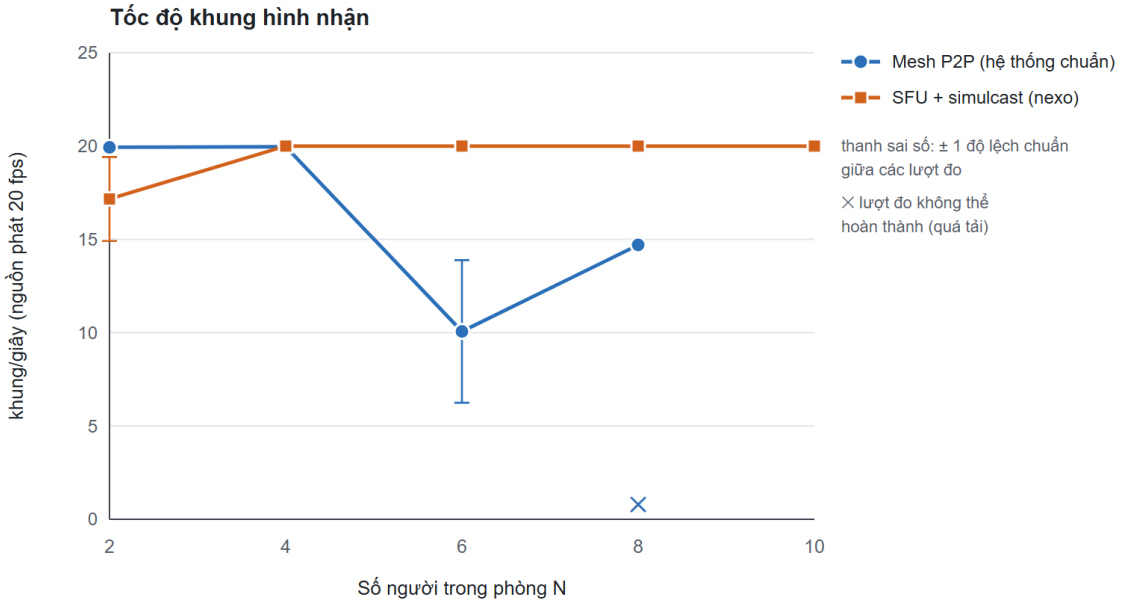
N	CPU worker mediasoup (% một lõi)	CPU Node.js (% một lõi)	RSS Node.js (MiB)	Tổng downlink mọi client (Mbps, tính lại)
2	2,7 ± 0,5	0,1 ± 0,1	86 ± 0	1,0
4	10,3 ± 1,3	0,2 ± 0,1	92 ± 0	3,8
6	19,7 ± 0,9	0,2 ± 0,1	98 ± 0	9,4
8	26,6 ± 0,8	0,3 ± 0,1	102 ± 1	9,7
10	38,9 ± 2,8	1,0 ± 0,5	109 ± 0	15,4

Cộng CPU của mọi trình duyệt cho tổng tải của thực nghiệm (*tính lại* bằng N × CPU trung bình mỗi client): Mesh dùng tổng cộng khoảng 324% một lõi ở N = 4, 1.213% ở N = 6 và 1.232% ở N = 8, trong khi SFU dùng 135% ở N = 4 và 471% ở N = 10, cộng thêm 38,9% cho worker. Mesh ở N = 6 và 8 dùng phần lớn 16 luồng phần cứng của máy; vì vậy một phần sự sụp đổ của Mesh là do chính máy thử nghiệm, điểm này được thảo luận ở mục 5.7.

5.4.5 Kết quả: độ trễ và chất lượng



Hình 5.6. Thời gian khứ hồi theo số người tham gia (thang logarit)



Hình 5.7. Tốc độ khung hình nhận theo số người tham gia

Hình 5.6 và Hình 5.7 trình bày độ trễ và tốc độ khung hình. Với SFU, RTT nằm trong khoảng 0,19 đến 1,00 ms, jitter video tối đa 1,83 ms, jitter âm thanh tối đa 1,06 ms, tỉ lệ mất gói 0,00% và không có NACK ở mọi mức; từ N = 4 trở lên, tốc độ khung hình nhận luôn đạt 20,0 fps, bằng tốc độ của nguồn phát. Ngoại lệ là N = 2 với trung bình 17,2 ± 2,3 fps: ở mọi lượt, ít nhất một trong hai người nhận có một phần thời gian đo (7 đến 20 trong 30 mẫu) dưới 18 fps, thấp nhất 5 fps, và downlink cũng thấp và dao động hơn tương ứng. Nhiều khả năng đây là lúc bộ ước lượng băng thông vẫn đang tăng dần tới lớp độ phân giải đầy đủ, lớp mà chính sách chọn khi chỉ có một camera từ xa và cần bitrate lớn nhất; hiện tượng này không xảy ra ở các mức N lớn hơn, nơi dùng lớp thấp hơn. Giữ phòng một camera ở lớp 1 cho tới khi ước lượng băng thông đủ lớn sẽ loại bỏ được hiện tượng này. Không có lần đóng băng hình nào trong cửa sổ đo của các lượt SFU; vài lần đóng băng ghi nhận lúc phòng đang đầy dần tương ứng với các lần chuyển lớp khi phòng vượt quá sáu camera.

Mesh đạt chất lượng tương đương ở N = 2 và 4 (RTT khoảng 1 ms, không mất gói, 20 fps) nhưng **sập ở N = 6**: RTT tăng lên 3.457 ± 850,4 ms, jitter buffer giữ 4.077 ± 3.833 ms dữ liệu, tốc độ khung hình nhận giảm còn 10,1 ± 3,8 fps, jitter âm thanh lên tới 90,92 ms, mỗi client gặp 31 lần đóng băng hình mỗi phút, và trình duyệt báo

chất lượng gửi bị giới hạn bởi băng thông hoặc CPU trong 27% số mẫu. Ở N = 8, một trong ba lượt thậm chí không thể hoàn thành vì các trình duyệt ngừng phản hồi harness, và các lượt hoàn thành có RTT lên tới vài giây. Khung hình đầu tiên xuất hiện sau 5.181 ms ở N = 2 và 10.648 ms ở N = 8 với Mesh, so với 6.228–12.719 ms với SFU; với SFU, thời gian này bao gồm cả việc mọi người lần lượt tham gia qua API.

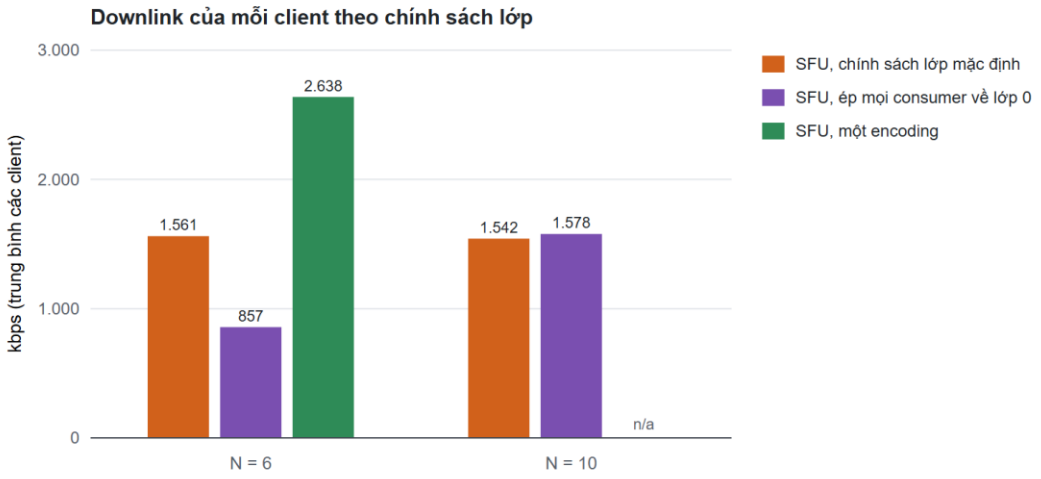
5.4.6 Tác dụng của simulcast và chọn lớp

Bảng 5.9 và Hình 5.8 tách riêng đóng góp của simulcast và của chính sách chọn lớp. Với **một bản mã hóa**, client SFU chỉ tải lên 526 kbps ở N = 2, ngang client Mesh, nhưng mọi người nhận đều nhận luồng độ phân giải đầy đủ, nên downlink tăng tuyến tính tới 3.669 kbps ở N = 8, gấp 3,0 lần downlink của cấu hình simulcast ở cùng N (*tính lại*), và CPU client tăng lên 70,5% vì mọi luồng đều được giải mã ở độ phân giải đầy đủ. Như vậy simulcast đánh đổi một chi phí uplink cố định để có downlink và chi phí giải mã bị chặn trên. **Ép mọi consumer về lớp 0** làm downlink ở N = 6 giảm từ 1.561 kbps (lớp 1 theo chính sách mặc định) xuống 857 kbps, tức giảm 45% (*tính lại*); ở N = 10, chính sách mặc định vốn đã chọn lớp 0 nên hai cấu hình cho downlink gần như nhau (1.542 và 1.578 kbps). Lưới video theo nhu cầu ở mục 4.4.3 tinh chỉnh chính sách này theo từng ô trong giao diện thật: tạm dừng các ô ngoài trang và chỉ yêu cầu lớp lớn cho các ô lớn.

Bảng 5.9. Các biến thể SFU: simulcast với chính sách mặc định, một bản mã hóa, và ép lớp 0 (trung bình giữa các lượt)

Cấu hình	N	Uplink (kbps)	Downlink (kbps)	CPU (%)	FPS
Simulcast, chính sách mặc định	2	1.020	481	25,8	17,2
Simulcast, chính sách mặc định	6	989	1.561	41,7	20,0
Simulcast, chính sách mặc định	8	992	1.218	40,9	20,0
Simulcast, chính sách mặc định	10	992	1.542	47,1	20,0
Một encoding	2	526	533	23,5	20,0
Một encoding	4	524	1.590	32,5	20,0
Một encoding	6	523	2.638	48,4	20,0

Cấu hình	N	Uplink (kbps)	Downlink (kbps)	CPU (%)	FPS
Một encoding	8	519	3.669	70,5	20,0
Simulcast, ép lớp 0	6	999	857	32,0	20,0
Simulcast, ép lớp 0	10	985	1.578	48,5	20,0



Hình 5.8. Downlink của mỗi client với ba cấu hình SFU ở N = 6 và N = 10 (cấu hình một bản mã hóa chỉ đo tới N = 8)

5.4.7 Thảo luận

1. **Mesh sụp đổ trong khoảng N = 4 đến N = 6, trong khi SFU không có dấu hiệu suy giảm tới N = 10.** Kết quả này khớp với Bảng 2.2: chi phí của Mesh tăng tuyến tính theo N−1 ở cả uplink, downlink và CPU, còn SFU giữ uplink không đổi và để chính sách chọn lớp kiểm soát downlink.
2. **SFU không rẻ hơn ở mọi N.** Ở N = 2, simulcast tốn nhiều uplink hơn một kết nối Mesh. Dùng một bản mã hóa, hoặc truyền ngang hàng cho cuộc gọi hai người, sẽ là lựa chọn tốt hơn trong trường hợp này; hệ thống chưa hiện thực việc tự động chuyển chế độ.
3. **Uplink SFU đo được (khoảng 1,0 Mbps) thấp hơn con số 1,75 Mbps trong ví dụ ở mục 2.3.4.** Con số đó là tổng bitrate tối đa của ba lớp. Với nội dung giả đơn giản ở 960×540 và 20 fps, bộ mã hóa không dùng hết

ngân sách; camera thật sẽ đẩy uplink lên gần mức trần hơn. Điều quan trọng là uplink **không** phụ thuộc vào N.

4. **Chi phí được chuyển sang máy chủ.** Băng thông ra của SFU bằng tổng downlink của mọi client (khoảng 15,4 Mbps ở N = 10, *tính lại*), và CPU của worker tăng nhanh hơn tuyến tính theo N. Máy chủ SFU phải được cấp đủ băng thông này, một chi phí mà Mesh không có.

5.5 So sánh với các hệ thống hiện có

Ngoài hệ thống chuẩn Mesh đã đo, Bảng 5.10 đặt nexo cạnh một số nền tảng phổ biến. Mô hình kênh lâu dài tích hợp thoại và video là cách làm của Discord và Slack [2], [1]; các sản phẩm chuyên về họp như Jitsi Meet [28] tập trung vào bản thân cuộc họp. Giống Jitsi Videobridge và các máy chủ media của Discord, nexo dùng bộ định tuyến chọn lọc, kiến trúc mà các nghiên cứu về SFU mã nguồn mở [23] và kết quả ở trên cho thấy là lựa chọn mở rộng được cho cuộc gọi nhiều bên. Cơ chế Last N của Jitsi [22] chỉ chuyển tiếp luồng của những người nói gần đây nhất; còn nexo xác định các luồng cần chuyển tiếp và lớp của chúng dựa trên những ô thực sự đang hiển thị, một chính sách bổ trợ có thể kết hợp với hoạt động nói trong hướng phát triển.

Bảng 5.10. So sánh định tính với các nền tảng hiện có

Tính năng	nexo	Discord	Slack	Jitsi Meet
Workspace với kênh văn bản lâu dài	✓	✓	✓	–
Gọi video ngay trong workspace	✓ (kênh video)	✓ (kênh thoại/video)	✓ (huddle)	– (cuộc họp riêng)
Kiến trúc media	SFU (mediasoup)	SFU (máy chủ riêng)	Không công bố	SFU (Jitsi Videobridge)
Tự triển khai, có mã nguồn	✓	–	–	✓
Phòng chờ và quyền điều hành của host	✓	Một phần	Một phần	✓
Bảng công việc liên kết với tin nhắn	✓	–	Qua tích hợp	–

Phép so sánh này mang tính định tính: các nền tảng thương mại chạy trên hạ tầng toàn cầu không thể tái tạo trên một máy, nên đồ án không đưa ra so sánh hiệu năng với chúng.

5.6 Đối chiếu với yêu cầu

Bảng 5.11 đối chiếu các yêu cầu phi chức năng ở Bảng 3.3 với kết quả đo.

Bảng 5.11. Đối chiếu các yêu cầu phi chức năng

Yêu cầu	Kết quả đo	Đánh giá
Tin nhắn tới người nhận dưới 100 ms (p95) trong mạng nội bộ	p95 nhận tin 32,2 / 34,9 / 45,9 ms ở 50 / 100 / 200 socket; p95 ACK tối đa 48,6 ms; p99 ở 200 socket khoảng 140 ms	**Đạt** theo tiêu chí p95 trên một máy; phần đuôi p99 ở 200 socket vượt 100 ms
Uplink và CPU client không tăng theo N (SFU)	Uplink 1.020 → 992 kbps khi N = 2 → 10; CPU 25,8% → 47,1%	**Đạt về uplink**; CPU tăng chậm theo số luồng giải mã, thấp hơn nhiều so với mức tăng tuyến tính của Mesh
Mất gói ≈ 0, jitter < 30 ms, RTT < 150 ms khi mạng tốt	SFU: mất gói 0%, jitter video ≤ 1,83 ms, jitter âm thanh ≤ 1,06 ms, RTT ≤ 1,00 ms tới N = 10	**Đạt** trên đường truyền nội bộ; chưa đo trên mạng thật
Không mất, không trùng tin khi mất ACK hoặc kết nối lại; phục hồi ICE	0 mất, 0 trùng trên 732 tin với 36 lần gửi lại và 6 lần kết nối lại; restart ICE đã hiện thực và kiểm thử ở mức signaling	**Đạt** trong các kịch bản đã kiểm thử
Băm mật khẩu, token ngắn hạn, kiểm quyền từng sự kiện, DTLS-SRTP	Kiểm thử phân quyền chéo đều đạt; DTLS-SRTP là bắt buộc trong WebRTC	**Đạt** với phân quyền; chưa có bắt gói để chứng minh DTLS-SRTP trên đường truyền
Nhiều tiến trình API; phân bổ cuộc họp theo worker	Hai tiến trình API và một worker bị dừng trong kiểm thử tích hợp	**Đạt** với nhiều tiến trình và worker trên một máy; SFU nhiều node chưa được nghiệm thu

5.7 Giới hạn của phép đo

Các giới hạn sau xác định phạm vi những gì có thể kết luận từ chương này.

1. **Một máy.** Mọi trình duyệt, API, PostgreSQL và SFU dùng chung 16 luồng phần cứng. Khi tổng nhu cầu CPU tiến gần năng lực của máy, mọi chỉ số đều xấu đi do tranh chấp. Mesh ở N = 6 và 8 rơi vào trạng thái này, nên một phần sự sụp đổ của nó đến từ máy thử; trên thiết bị thật, mỗi người có CPU riêng, và điểm gãy của Mesh do uplink hoặc CPU của thiết bị yếu nhất quyết định, có thể nằm ở một giá trị N khác. Kết luận vững chắc là xu hướng (Mesh tăng tuyến tính, uplink SFU phẳng), chứ không phải vị trí chính xác của điểm gãy.
2. **Mạng nội bộ.** Không có mất gói, độ trễ hay giới hạn băng thông thật, nên RTT, jitter và tỉ lệ mất gói phản ánh tải xử lý chứ không phản ánh chất lượng mạng. Chưa thực hiện thử nghiệm trên nhiều mạng, qua TURN hay với suy giảm mạng có kiểm soát.
3. **Media giả và không vẽ video.** Camera giả tạo nội dung đơn giản nên bitrate thấp hơn camera thật, và chi phí vẽ video đã bị loại khỏi phép đo. CPU thực tế sẽ cao hơn một lượng xấp xỉ nhau ở cả hai kiến trúc.
4. **Ba lượt mỗi mức.** Độ lệch chuẩn dựa trên ba lượt mỗi mức (hai lượt với Mesh ở N = 8, nơi một lượt không thể hoàn thành), đủ để thấy xu hướng nhưng chưa đủ để có khoảng tin cậy hẹp.
5. **Chưa bắt gói tin.** Công cụ phân tích và quy trình bắt gói đã có trong kho mã nhưng chưa thực hiện lần bắt gói nào, nên chưa có bằng chứng ở mức gói tin cho STUN, DTLS và RTP/RTCP.
6. **Tải nhắn tin ở mức vừa phải.** Bài kiểm tra nhắn tin dùng 4 tin mỗi giây; chưa mô phỏng chạy dài hạn, tải bão hòa, mất mạng kéo dài hay mạng di động.
7. **Hạ tầng chưa được nghiệm thu.** CI từ xa, sao lưu trên môi trường thật, S3 và e-mail trên đám mây, HTTPS công khai và máy chủ TURN thật nằm

ngoài môi trường thử nghiệm, nên hệ thống chưa được khẳng định là sẵn sàng cho môi trường sản xuất.

5.8 Tóm tắt chương

Các bài kiểm thử tự động xác nhận tính đúng của cơ chế lũy đẳng khi gửi tin, phân quyền, quản trị, khả năng chịu lỗi worker, hoạt động nhiều tiến trình, migration và khôi phục dữ liệu. Lõi nhắn tin giữ độ trễ nhận tin ở phân vị 95 dưới 46 ms với 200 socket và không làm mất hay trùng tin nào. Trong thực nghiệm chính, SFU giữ uplink của client ở khoảng 1 Mbps và CPU client ở mức thấp từ N = 2 tới N = 10, không mất gói và ổn định ở 20 fps từ N = 4 trở lên, trong khi hệ thống chuẩn Mesh tăng tuyến tính và sập ở N = 6 trên cùng máy. Các kết quả này thu được trên một máy qua mạng nội bộ; mở rộng phép đo sang mạng thật là hạng mục chính của hướng phát triển.

CHƯƠNG 6. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

6.1 Kết quả đạt được so với mục tiêu

Chương 1 đặt ra bốn mục tiêu. Bảng 6.1 nhắc lại từng mục tiêu, tóm tắt kết quả đạt được, cách kiểm chứng và mức độ hoàn thành.

Bảng 6.1. Mục tiêu ở Chương 1 và mức độ đạt được

Mục tiêu	Kết quả	Minh chứng	Mức độ
O1 – Không gian làm việc tích hợp	Ứng dụng web tổ chức theo Workspace → Channel với kênh văn bản, thoại và video, kênh riêng tư, lời mời, phân quyền, quản trị và nhật ký, kèm bảng công việc, thông báo, tìm kiếm và hồ sơ cá nhân	Chương 3–4; kiểm thử nghiệm thu trên trình duyệt; Hình 4.6–4.10	Đạt
O2 – Nhắn tin và trạng thái hiện diện thời gian thực	Gửi tin lũy đẳng (mã do client sinh, outbox IndexedDB, advisory lock và ràng buộc duy nhất), lấy bù tin sau khi kết nối lại, presence có TTL giữa các tiến trình, trạng thái đang gõ	Mục 5.2; mục 5.3: 0 tin mất, 0 tin trùng, p95 nhận tin ≤ 46 ms với 200 socket	Đạt trên một máy; chưa đo trên mạng diện rộng
O3 – Hội nghị truyền hình đa điểm	SFU mediasoup với nhóm worker, simulcast ba lớp, chọn lớp và tạm dừng theo nhu cầu, phục hồi ICE, thông tin xác thực TURN có thời hạn, phòng chờ và điều hành	Mục 5.4: uplink SFU 989–1.020 kbps từ N = 2 đến N = 10, không mất gói (20 fps từ N = 4); Mesh sập ở N = 6	Đạt trong phòng thí nghiệm; còn thiếu thử nghiệm nhiều mạng và TURN

Mục tiêu	Kết quả	Minh chứng	Mức độ
O4 – Đánh giá định lượng	Phòng thí nghiệm media với trình duyệt thật, lặp nhiều lượt và lưu dữ liệu gốc; kiểm tra tải lõi nhắn tin; so sánh với hệ thống chuẩn Mesh	Chương 5, dữ liệu gốc trong `docs/evidence/`	Đạt trên một máy; còn thiếu bắt gói tin và thử nghiệm nhiều máy

Luận điểm kỹ thuật trung tâm của đồ án, rằng SFU giữ chi phí của mỗi người tham gia không phụ thuộc vào quy mô phòng trong khi Mesh thì không, đã được kiểm chứng bằng thực nghiệm trên cùng phần cứng. Từ N = 2 đến N = 10, uplink của một client SFU nằm trong khoảng 989–1.020 kbps và CPU dưới 47% một lõi, trong khi CPU của một client Mesh tăng 3,1 lần từ N = 2 lên N = 4 và phiên Mesh sập ở N = 6.

6.2 Đóng góp của đồ án

Các đóng góp chính của đồ án gồm:

1. Một nền tảng cộng tác hoạt động được, có thể tự triển khai, tích hợp nhắn tin theo kênh lâu dài và hội nghị đa điểm trong cùng một ứng dụng trình duyệt, với một mô hình phân quyền thống nhất được áp dụng cho mọi yêu cầu REST và mọi sự kiện socket.
2. Một giao thức gửi tin kết hợp mã định danh do client sinh, outbox bền vững, advisory lock trong phạm vi giao dịch và ràng buộc duy nhất, để có ngữ nghĩa lưu đúng một lần trên nền truyền ít nhất một lần, đã được kiểm chứng với các trường hợp mất acknowledgement, gửi lại đồng thời và kết nối lại.
3. Một lưới video theo nhu cầu, suy ra lớp simulcast của từng consumer từ kích thước ô hiển thị và tạm dừng các consumer không hiển thị, để downlink đi theo những gì người dùng nhìn thấy thay vì theo quy mô phòng.

4. Một phòng thí nghiệm media có thể chạy lại, trong đó mỗi người tham gia là một trình duyệt thật, so sánh SFU với hệ thống chuẩn Mesh trong cùng điều kiện, lặp lại từng mức và lưu toàn bộ mẫu gốc, kèm ghi chép các sai lệch đo lường (công bố địa chỉ loopback và gói thăm dò băng thông) mà nếu bỏ qua sẽ làm sai kết quả.

6.3 Hạn chế

- Mọi phép đo media và nhắn tin đều thực hiện trên một máy, qua đường nội bộ tới địa chỉ LAN của chính máy đó. Kết quả phản ánh tin cậy xu hướng và chi phí tương đối, nhưng chưa đo hành vi khi có độ trễ, mất gói hay giới hạn băng thông thật.
- Chưa bắt gói tin, nên hoạt động của DTLS-SRTP và RTCP mới được chứng minh qua đặc tả và qua `getStats()`, chưa có bằng chứng ở mức gói tin.
- Camera giả tạo nội dung đơn giản; camera thật sẽ cho bitrate cao hơn, và chi phí vẽ video đã bị loại khỏi số đo CPU.
- Mô hình SFU nhiều node, TURN trên máy chủ công khai, gửi e-mail thật, lưu trữ đám mây, CI từ xa và sao lưu trên môi trường thật đã được hiện thực hoặc cấu hình nhưng chưa được nghiệm thu trên hạ tầng thật. Vì vậy hệ thống chưa được khẳng định là sẵn sàng cho môi trường sản xuất.
- Chưa có ứng dụng di động gốc, chưa ghi hình cuộc họp và chưa có mã hóa đầu-cuối xuyên qua SFU.

6.4 Hướng phát triển

1. **Đánh giá trên nhiều mạng.** Lặp lại thực nghiệm media với client trên nhiều máy và nhiều mạng, có một client buộc phải đi qua TURN, và với suy giảm mạng có kiểm soát (mất gói, độ trễ, giới hạn băng thông) tại bộ định tuyến; bắt gói tin bằng Wireshark để phân tích STUN, DTLS, RTP và RTCP (SR/RR, NACK, PLI) và đối chiếu với `getStats()`.

2. **Tự chọn cấu trúc kết nối cho phòng nhỏ.** Dùng một bản mã hóa hoặc truyền ngang hàng cho cuộc gọi hai người, nơi simulcast tốn uplink hơn lợi ích mang lại, và chuyển sang SFU khi người thứ ba tham gia.
3. **Chọn luồng theo người đang nói.** Kết hợp lưới video theo nhu cầu với việc chọn luồng chuyển tiếp dựa trên mức âm thanh (theo tinh thần của Last N [22]) cho các phòng lớn.
4. **Mở rộng tầng media.** Nối router giữa các worker và các node (pipe) để một cuộc họp rất lớn có thể trải trên nhiều worker, và nghiệm thu mô hình nhiều node với cân bằng tải trên máy chủ thật.
5. **Tính năng sản phẩm.** Tin nhắn trực tiếp giữa hai người, ghi hình cuộc họp, mã hóa đầu-cuối bằng insertable streams, và ứng dụng di động gốc hoặc ứng dụng web tiến bộ.

Tóm lại, đồ án đã xây dựng được một không gian làm việc thời gian thực tích hợp hội nghị đa điểm ngay trên trình duyệt, và chứng minh bằng các phép đo có thể lặp lại rằng kiến trúc media dựa trên SFU của hệ thống mở rộng được ở những quy mô mà hệ thống chuẩn ngang hàng không đáp ứng nổi. Phần việc còn lại chủ yếu là mở rộng phạm vi đánh giá từ một máy sang mạng thật và hạ tầng thật.

TÀI LIỆU THAM KHẢO

[1] Slack Technologies, "What is a channel?," Slack Help Center. [Online]. Available: https://slack.com/help/articles/360017938993-What-is-a-channel (accessed Oct. 2026).

[2] J. Vass, "How Discord handles two and half million concurrent voice users using WebRTC," Discord Engineering Blog, Sep. 2018. [Online]. Available: https://discord.com/blog/how-discord-handles-two-and-half-million-concurrent-voice-users-using-webrtc

[3] Meta Open Source, "React documentation." [Online]. Available: https://react.dev (accessed Oct. 2026).

[4] Socket.IO, "Socket.IO v4 documentation." [Online]. Available: https://socket.io/docs/v4/ (accessed Oct. 2026).

[5] I. Baz Castillo and J. L. Millán, "mediasoup v3 documentation." [Online]. Available: https://mediasoup.org/documentation/v3/ (accessed Oct. 2026).

[6] I. Fette and A. Melnikov, "The WebSocket Protocol," IETF RFC 6455, Dec. 2011.

[7] C. Jennings, F. Castelli, H. Boström and J.-I. Bruaroey, "WebRTC: Real-Time Communication in Browsers," W3C Recommendation. [Online]. Available: https://www.w3.org/TR/webrtc/

[8] H. Boström, V. Singh and J.-I. Bruaroey, "Identifiers for WebRTC's Statistics API," W3C Candidate Recommendation. [Online]. Available: https://www.w3.org/TR/webrtc-stats/

[9] A. Begen, P. Kyzivat, C. Perkins and M. Handley, "SDP: Session Description Protocol," IETF RFC 8866, Jan. 2021.

[10] J. Rosenberg and H. Schulzrinne, "An Offer/Answer Model with the Session Description Protocol (SDP)," IETF RFC 3264, Jun. 2002.

[11] J. Uberti, C. Jennings and E. Rescorla, "JavaScript Session Establishment Protocol (JSEP)," IETF RFC 8829, Jan. 2021.

[12] A. Keranen, C. Holmberg and J. Rosenberg, "Interactive Connectivity Establishment (ICE): A Protocol for Network Address Translator (NAT) Traversal," IETF RFC 8445, Jul. 2018.

[13] M. Petit-Huguenin, G. Salgueiro, J. Rosenberg, D. Wing, R. Mahy and P. Matthews, "Session Traversal Utilities for NAT (STUN)," IETF RFC 8489, Feb. 2020.

[14] T. Reddy, A. Johnston, P. Matthews and J. Rosenberg, "Traversal Using Relays around NAT (TURN): Relay Extensions to Session Traversal Utilities for NAT (STUN)," IETF RFC 8656, Feb. 2020.

[15] J. Uberti, "A REST API for Access to TURN Services," IETF Internet-Draft draft-uberti-behave-turn-rest-00, Mar. 2013.

[16] D. McGrew and E. Rescorla, "Datagram Transport Layer Security (DTLS) Extension to Establish Keys for the Secure Real-time Transport Protocol (SRTP)," IETF RFC 5764, May 2010.

[17] M. Baugher, D. McGrew, M. Naslund, E. Carrara and K. Norrman, "The Secure Real-time Transport Protocol (SRTP)," IETF RFC 3711, Mar. 2004.

[18] H. Schulzrinne, S. Casner, R. Frederick and V. Jacobson, "RTP: A Transport Protocol for Real-Time Applications," IETF RFC 3550, Jul. 2003.

[19] S. Holmer, H. Lundin, G. Carlucci, L. De Cicco and S. Mascolo, "A Google Congestion Control Algorithm for Real-Time Communication," IETF Internet-Draft draft-ietf-rmcat-gcc-02, Jul. 2016.

[20] J. Bankoski, J. Koleszar, L. Quillio, J. Salonen, P. Wilkins and Y. Xu, "VP8 Data Format and Decoding Guide," IETF RFC 6386, Nov. 2011.

[21] J.-M. Valin, K. Vos and T. Terriberry, "Definition of the Opus Audio Codec," IETF RFC 6716, Sep. 2012.

[22] B. Grozev, L. Marinov, V. Singh and E. Ivov, "Last N: Relevance-based selectivity for forwarding video in multimedia conferences," in Proc. 25th ACM Workshop on Network and Operating Systems Support for Digital Audio and Video (NOSSDAV), Portland, OR, USA, 2015.

[23] E. André, N. Le Breton, A. Lemesle, L. Roux and A. Gouaillard, "Comparative study of WebRTC open source SFUs for video conferencing," in Proc. Principles, Systems and Applications of IP Telecommunications (IPTComm), Chicago, IL, USA, 2018.

[24] R. S. Sandhu, E. J. Coyne, H. L. Feinstein and C. E. Youman, "Role-based access control models," IEEE Computer, vol. 29, no. 2, pp. 38–47, Feb. 1996.

[25] C. Percival and S. Josefsson, "The scrypt Password-Based Key Derivation Function," IETF RFC 7914, Aug. 2016.

[26] M. Jones, J. Bradley and N. Sakimura, "JSON Web Token (JWT)," IETF RFC 7519, May 2015.

[27] PostgreSQL Global Development Group, "PostgreSQL 17 documentation: NOTIFY, LISTEN and advisory locks." [Online]. Available: https://www.postgresql.org/docs/17/ (accessed Oct. 2026).

[28] Jitsi, "Jitsi Meet handbook: architecture of Jitsi Meet and Jitsi Videobridge." [Online]. Available: https://jitsi.github.io/handbook/ (accessed Oct. 2026).